

University College Dublin



# CAPM Assignment

Individual Practical Assignment In Python

James Murphy

FIN30200

13<sup>th</sup> October 2024

# I. Introduction

A fundamental idea of modern finance is that an investor needs a financial incentive to take a risk. In other words, the expected return on a risky investment,  $R$ , must exceed the return on a safe, or risk-free, investment,  $R_f$ . Thus the expected excess return,  $R - R_f$ , on a risky investment, like owning stock in a company, should be positive. At first, it might seem like the risk of a stock should be measured by its variance. Much of that risk, however, can be reduced by holding other stock in a "portfolio" (i.e. by diversifying your financial holdings). This means that the right way to measure the risk of a stock is not by its variance but rather by its covariance with the market. The Capital Asset Pricing Model (CAPM) formalises this idea. According to this model, the expected excess return on an asset is proportional to the expected excess return on a portfolio of all available assets ("the market portfolio").

The CAPM says:

$$(R_{it} - R_{ft}) = \alpha_j + \beta_j(R_{mt} - R_{ft})$$

where  $R_{it}$  is the expected return of the firm  $j$ ,  $R_{ft}$  measures the expected return of a risk-free asset during period  $t$  (the Federal Funds Rate, FFR), and  $R_{mt}$  is the expected return on the market portfolio during period  $t$  (SP500).

According to the CAPM, a stock with a  $\beta < 1$  has less risk than the market portfolio and therefore has a lower expected excess return than the market portfolio. Meanwhile, a stock with  $\beta > 1$  is riskier than the market portfolio and thus commands a higher expected excess return.

The objective of this assignment is to evaluate the effectiveness of the CAPM using two cryptocurrencies, Bitcoin (BTC) and Ethereum (ETH). To perform this analysis, price data for BTC and ETH was obtained from Yahoo Finance, using weekly data ending each Sunday based on the last available price. Data with respect to the S&P 500 index and risk-free rate were obtained from the Federal Reserve of St. Louis (FRED), also on a weekly, Sunday-ending basis.

## II. Linear Regression Using Ordinary Least Squares

### i. Transformation of Data

Before we perform linear regression, we must first transform our datasets such that we obtain the log weekly returns for our two cryptocurrencies and the SP500, where

$$R_{i,t} = 100\% \times \log\left(\frac{P_{i,t}}{P_{i,t-1}}\right)$$

There are several reasons why we use log transformations, some of which are mentioned below (Wooldridge (2010)).

**Linearising Relationships:** Asset prices often grow exponentially, particularly in financial markets. Taking the log of prices transforms this exponential growth into a linear form, making it easier to apply OLS regression in CAPM. By using log-returns (i.e.,  $\log\left(\frac{P_t}{P_{t-1}}\right)$ ), we can model the relationship between the market returns and asset returns more effectively, as CAPM assumes a linear relationship between these variables.

**Stabilising Variance (Reducing Heteroscedasticity):** Returns of financial assets, particularly highly volatile assets like BTC and ETH), often exhibit heteroscedasticity; where the variability of returns changes over time. This violates one of the assumptions of OLS regression. Log transformations often stabilise the variance of the residuals, improving the reliability of the estimated coefficients in CAPM.

**Addressing Skewness and Outliers:** Financial return distributions can be skewed or have extreme values. Log transformations reduce skewness and compress the range of large values, making the data more symmetric and normally distributed. This improves the model's fit and aligns better with the normality assumptions of OLS regression used in CAPM.

## ii. Simple Linear Regression

Now that we have transformed our data, we can perform a simple linear regression model to assess the effectiveness of the CAPM. Simple Linear Regression is a basic yet powerful statistical method used to model the relationship between two variables: one independent variable (also called the predictor or explanatory variable) and one dependent variable (also known as the response or outcome variable). The goal of this method is to predict the value of the dependent variable based on the value of the independent variable. In the case of this assignment, we want to model the relationship between the log weekly excess returns of the market and the log weekly excess returns of a cryptocurrency. The formula for the simple linear regression we will be performing is:

$$y = \beta_0 + \beta_1 x + \epsilon$$

where:

$y$ : Dependent variable ( $R_{it} - R_{ft}$ )

$x$ : Independent variable ( $R_{mt} - R_{ft}$ )

$\beta_0$ : Intercept

$\beta_1$ : Slope

$\epsilon$ : Error term (or residual)

The slope  $\beta_1$  can be computed using the formula:

$$\beta_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

Similarly, we can compute:

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

where  $\bar{y}$  is the mean of  $y$  and  $\bar{x}$  is the mean of  $x$ .

Finally, we can calculate our residual value

$$\epsilon_i = y_i - \hat{y}_i$$

where  $\hat{y}_i = \beta_0 + \beta_1 x_i$ .

The classical linear regression model (CLRM) above is dependent on a number of key assumptions (Brooks, 2008):

### 1) The Errors have Zero Mean

This assumption means that, on average, the error term  $\epsilon_i$  is expected to be zero. In other words, the model is correctly specified and no systematic bias exists in the residuals.

$$E[\epsilon] = 0$$

### 2) Variance of Errors is Constant and Finite

This is the assumption of homoscedasticity, which means that the variance of the error term  $\epsilon_i$  is the same for all values of the independent variable  $x_i$ . Constant variance ensures that the reliability of the coefficient estimates remains stable across observations.

$$\text{Var}(\epsilon) = \sigma^2 < \infty$$

### 3) The Errors are Linearly Independent of One Another

This is the assumption of no autocorrelation, which means that the errors for different observations are not correlated with each other. In time series data, this implies that the error at time  $i$  is not related to the error at time  $j$ .

$$\text{Cov}(\epsilon_i, \epsilon_j) = 0, \quad i \neq j$$

### 4) No Relationship between the Error and the Independent Variable

This assumption is known as exogeneity and ensures that the independent variables  $x_i$  are not correlated with the error term  $\epsilon_i$ . If violated, the regression estimates would be biased and inconsistent.

$$Cov(\epsilon_i, x_i) = 0$$

## 5) The Errors are Normally Distributed

This assumption means that the errors follow a normal distribution with a mean of zero and variance  $\sigma^2$ . This assumption is very important for conducting hypothesis tests and deriving confidence intervals, especially when working with small sample sizes.

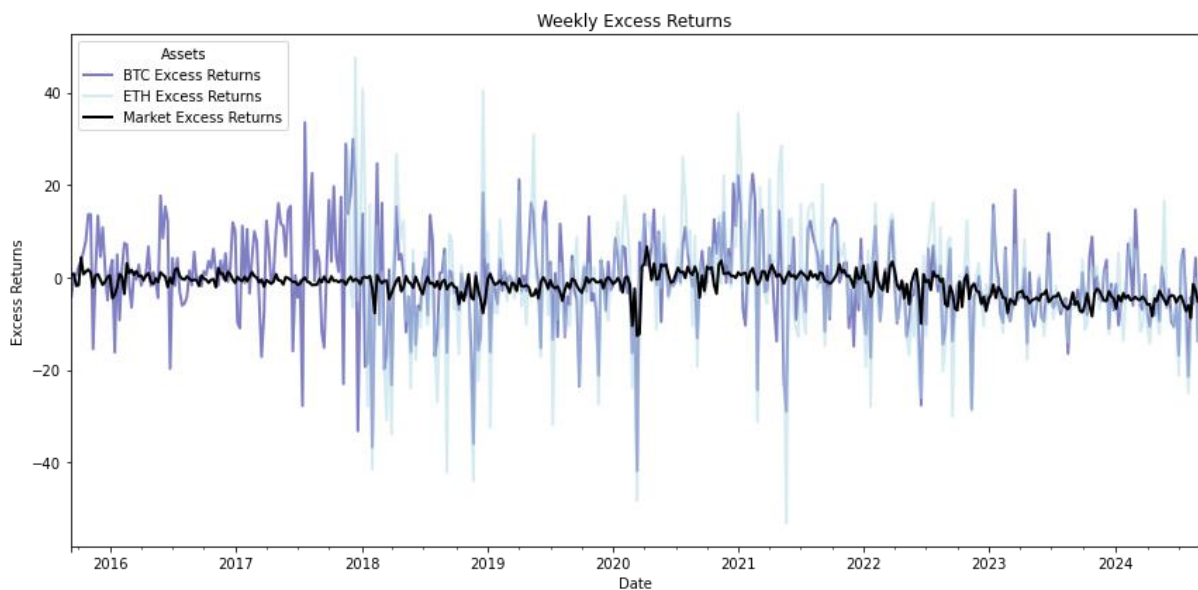
$$\epsilon_i \sim N(0, \sigma^2)$$

These assumptions are crucial for ensuring that the OLS estimators are BLUE (Best Linear Unbiased Estimators) under the Gauss-Markov theorem.

### ii.i. Defining Independent and Dependent Variables

Now that we have completed log transformations of our data and introduced classical linear regression, we can calculate both the independent and dependent variables for our model. We can calculate the independent variable  $(R_{mt} - R_{ft})$ , by subtracting the risk-free rate of return (Federal Funds Effective Rate) from the return of the SP500 at each weekly timepoint. Similarly, we can calculate our dependent variable  $(R_{it} - R_{ft})$  by subtracting the weekly risk-free rate of return from the weekly return on each cryptocurrency, BTC and ETH.

Figure 1: Weekly Log Excess Returns



## ii.ii. Model Summary

Table 1: Simple Linear Regression Results

|                    | BTC       |           | ETH       |           |
|--------------------|-----------|-----------|-----------|-----------|
|                    | $\beta_0$ | $\beta_1$ | $\beta_0$ | $\beta_1$ |
| Coefficient        | 1.1738    | 1.1436    | 1.4391    | 1.5398    |
| Std. Error         | 0.522     | 0.172     | 0.806     | 0.236     |
| R-Squared          | 0.086     |           | 0.107     |           |
| Adjusted R-Squared | 0.084     |           | 0.105     |           |
| F-Statistic        | 44.14     |           | 42.48     |           |
| Prob (F-Statistic) | 8.52e-11  |           | 2.46e-10  |           |
| Log-Likelihood     | -1728.4   |           | -1396.0   |           |
| AIC                | 3461.     |           | 2796.     |           |
| BIC                | 3469.     |           | 2804.     |           |
| Omnibus            | 19.837    |           | 33.407    |           |
| Prob (Omnibus)     | 0.000     |           | 0.000     |           |
| Skew               | -0.211    |           | -0.264    |           |
| Kurtosis           | 4.380     |           | 5.905     |           |
| Durbin-Watson      | 1.934     |           | 1.930     |           |
| Jarque-Bera (JB)   | 40.723    |           | 128.966   |           |
| Prob (JB)          | 1.44e-09  |           | 9.89e-29  |           |

Table 2: Significance Results for BTC

Significance Test for BTC

| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| Intercept ( $\beta_0$ )   | 2.249 | 1.96           | 0.025   | [0.148              | 2.200] |
| Coefficient ( $\beta_1$ ) | 6.644 | 1.96           | 0.000   | [0.805              | 1.482] |

Table 3: Significance Results for ETH

Significance Test for ETH

| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| Intercept ( $\beta_0$ )   | 1.787 | 1.96           | 0.075   | [-0.145             | 3.023] |
| Coefficient ( $\beta_1$ ) | 6.518 | 1.96           | 0.000   | [1.075              | 2.004] |

## ii.iii. Assessing Statistical Significance

Now that we have obtained the results for our models, we can perform various statistical significance tests for the coefficient estimates. The goal is to determine whether the relationships between the independent variables ( $R_{mt} - R_{ft}$ ) and the dependent variables ( $R_{it} - R_{ft}$ ) are statistically significant.

## 1. t-Test for Coefficients

Null Hypothesis ( $H_0$ ):

$H_0: \beta = 0$  (The coefficient is zero, meaning the independent variable has no effect on the dependent variable).

Alternative Hypothesis ( $H_1$ )

$H_1: \beta \neq 0$  (The coefficient is significantly different from zero, meaning the independent variable has a meaningful effect).

The test statistic is calculated using:

$$t = \frac{\hat{\beta} - 0}{SE(\hat{\beta})}$$

Where  $\hat{\beta}$  is the estimated coefficient and  $SE(\hat{\beta})$  is its standard error.

The p-value associated with the t-statistic tells us whether or not we can reject the null hypothesis. A p-value less than 0.05 indicates that the coefficient is statistically significant at the 5% level.

## 2. Confidence Intervals for Coefficients

A confidence interval provides a range of plausible values for each coefficient, helping to assess its precision and significance. If the confidence interval does not include zero, our estimate for the coefficient ( $\hat{\beta}$ ) is considered statistically significant. The formula for constructing a confidence interval is:

$$CI = \hat{\beta} \pm z_{\alpha/2} \cdot SE(\hat{\beta})$$

Where  $z_{\alpha/2}$  represents the critical value from the standard normal distribution corresponding to the tail probability of  $\alpha/2$ . For example, in a 95% confidence interval ( $\alpha = 0.05$ ),  $z_{\alpha/2}$  equals 1.96, meaning that 95% of the values lie within 1.96 standard deviations of the mean.

## 3. p-value Test

The p-value test is one of the most widely used methods for assessing the statistical significance of an estimate. It helps determine whether the observed result is due to chance or if it reflects a real effect in the data. A p-value represents the probability of observing the test statistic (or something more extreme) given that the null hypothesis ( $H_0$ ) is true. In the context of regression, it helps answer the question: What is the likelihood of observing the

estimated coefficient if there were no real relationship between the independent variable and the dependent variable?

In a p-value test, the null and alternative hypothesis are the same as those in a t-Test. For our chosen  $\alpha$ , ( $\alpha = 0.05$  is very common), if  $p \leq \alpha$ , we reject the null hypothesis, meaning the result is statistically significant. Conversely, if  $p > \alpha$ , we fail to reject the null hypothesis, which means the result is not statistically significant and is therefore insufficient evidence to conclude that the independent variable affects the dependent variable.

### iii.iv. Interpretation of Models

#### **BTC Model:**

For our BTC model, we obtained a value of  $\beta_0 = 1.1738$  for our intercept. In the context of the CAPM, this reflects the abnormal returns of the cryptocurrency that are not explained by the market's excess returns. In simple terms, this value represents the weekly log excess return of the cryptocurrency when the weekly log excess return of the market is zero. As the intercept is positive, it signifies that one can expect BTC to outperform the market when the market provides no excess return. For the coefficient of our independent variable ( $R_{mt} - R_{ft}$ ), we obtained a value of  $\beta_1 = 1.1436$ . This value quantifies the systematic risk of the asset, which is the asset's sensitivity to market movements. A  $\beta_1$  of 1.1436 indicates that BTC's weekly log excess returns are more volatile and higher risk compared to the market. When the market's excess returns increase by 1%, BTC's returns would be expected to increase by 1.1436% on average. Similarly, if the market's returns fall by 1%, BTC's returns would be expected to decrease by 1.1436%. In summary, this implies that BTC is more volatile than the market and will likely experience larger swings in returns compared to an asset with a  $\beta_1$  of 1. Investors holding BTC, should therefore expect higher potential rewards, but also face higher risks, since the asset amplifies market movements.

We can now assess the significance of our results. For our intercept, we can see in Table 2 that the test statistic of 2.249 is greater than the critical value of 1.96 ( $\alpha = 0.05$ ) and the p-value of 0.025 is less than the significance level of 0.05. This indicates that the  $\beta_0$  is statistically significant at the 5% level, meaning that there is evidence of abnormal returns for BTC that are not explained by the market risk. The confidence interval for the intercept is [0.148, 2.200], which does not include zero. This further supports the significance of the intercept, implying that BTC has returns beyond what is predicted by market risk, suggesting some degree of alpha (abnormal performance) relative to the market. In the context of CAPM, a significant intercept could indicate that other factors (beyond market risk) are influencing BTC's returns. It must be noted, however, that a  $\beta_0 \neq 0$  violates an assumption of the CAPM. For  $\beta_1$ , the test statistic of 6.644 is much greater than the critical value of 1.96, and the p-value of 0.000 is far below the significance threshold of 0.05. This indicates that our estimate for  $\beta_1$  is also statistically significant at the 1% level, meaning the excess market return has a strong and significant relationship with BTC's excess returns. The confidence interval for  $\beta_1$  is [0.805, 1.482], which does not include zero, further confirming the statistical significance of this coefficient.



## ETH Model

For our Ethereum model, we obtained an intercept value of  $\beta_0=1.4391$ . In the context of the CAPM, this represents the abnormal returns of ETH that are not explained by the market's excess returns. In simpler terms, this value reflects the weekly log excess return of ETH when the market's weekly log excess return is zero. Since the intercept is positive, it indicates that ETH is expected to outperform the market when the market delivers no excess return. For the coefficient of our independent variable ( $R_{mt} - R_{ft}$ ), we obtained a value of  $\beta_1=1.5398$ . This coefficient measures the systematic risk of ETH, reflecting its sensitivity to market movements. A  $\beta_1$  of 1.5398 suggests that ETH's weekly log excess returns are significantly more volatile and risky compared to the market. When the market's excess returns increase by 1%, ETH's returns would be expected to increase by 1.5398% on average. Similarly, if the market's returns decrease by 1%, ETH's returns would be expected to fall by 1.5398%.

With respect to the significance of our results for Ethereum, we can see in Table 3 that for our intercept,  $\beta_0$ , the test statistic of 1.787 is less than the critical value of 1.96, and the p-value of 0.075 is greater than 0.05, meaning  $\beta_0$  is not statistically significant at the 5% significance level. This implies that there is insufficient evidence to conclude that the intercept is different from zero. This is, however, in line with the CAPM assumption that the intercept is equal to 0. Additionally, the confidence interval for the intercept is [-0.145, 3.023], which includes zero, reinforcing that the intercept is not statistically significant. For  $\beta_1$ , the test statistic of 6.518 is again much greater than the critical value of 1.96, and the p-value of 0.000 means that our result is statistically significant at the 1% level. The confidence interval for  $\beta_1$  is [1.075, 2.004], which does not include zero.

### iii.v. Further Statistical Inference

There are a number of additional statistical indicators that we can use for inference. A number of these statistical indicators will be re-used again in later parts of this assignment.

## F-Test for Overall Significance

While the t-test focuses on testing the significance of individual coefficients, the F-test evaluates the overall significance of a regression model. It can be used to assess whether the group of independent variables, taken together, significantly predicts the dependent variable. In other words, it helps answer the question: Does the model, as a whole, significantly predict the dependent variable?

When performing a F-statistic test, it is common practice to examine two models, one which is restricted and one which is unrestricted. The restricted model imposes constraints (such as setting certain coefficients to zero or limiting the number of predictors), making it a simpler version of the model, while the unrestricted model allows all coefficients to vary freely, providing a more flexible and complex representation of the data.

Null Hypothesis ( $H_0$ ):

$$H_0: \beta_1 = \beta_2 = \dots = \beta_k = 0$$

where  $k$  is the number of restrictions. This means that the restricted model (typically with fewer parameters or specific constraints) fits the data just as well as the unrestricted model (with more parameters or fewer constraints)

#### Alternative Hypothesis ( $H_1$ )

$H_1$ : At least one  $\beta_i \neq 0$

This means that the unrestricted model (with more parameters) fits the data significantly better than the restricted model, and the restrictions imposed in the restricted model lead to a worse fit.

The F-statistic is calculated by comparing the variance explained by the model to the variance that is unexplained (i.e. the variance due to errors). Mathematically:

$$F = \frac{(RSS_R - RSS_U)/m}{(RSS_U)/(T - k)}$$

where:

$RSS_R$ : The Residual Sum of Squares for the restricted model.

$RSS_U$ : The Residual Sum of Squares for the unrestricted model.

$m$ : The number of restrictions (the difference in the number of parameters between the unrestricted and restricted models).

$T - k$ : The degrees of freedom where  $T$  is the total number of observations and  $k$  is the number of parameters in the unrestricted model, including the intercept.

If the F-statistic is large and the p-value is small (typically less than 0.05), you reject the null hypothesis. This means the unrestricted model is significantly better than the restricted model, and the restrictions are not valid.

Likewise, if the F-statistic is small and the p-value is large (greater than 0.05), you fail to reject the null hypothesis. This means the restricted model fits the data as well as the unrestricted model, so the restrictions are valid.

In our BTC model, we obtained an F-statistic of 44.14, which at a confidence level of  $\alpha = 0.05$  corresponds to a critical value of 3.86 with numerator degrees of freedom ( $df_1$ ) = 1 and denominator degrees of freedom ( $df_2$ ) = 467. The fact that our F-statistic of 44.14 is much larger and corresponds to an extremely small p-value of 8.52e-11 indicates that the model is statistically significant. This means that the market's excess returns are a significant predictor of BTC's excess returns. The F-test result allows us to reject the null hypothesis and conclude that the market does have a meaningful effect on BTC returns, though other factors not captured by CAPM may also contribute.

In our ETH model, the F-statistic is 42.48, which at a confidence level of  $\alpha = 0.05$  corresponds to a critical value of 3.87 with numerator degrees of freedom ( $df_1$ ) = 1 and denominator degrees of freedom ( $df_2$ ) = 353. Our F-statistic of 42.48 is larger and corresponds to a p-value of 2.46e-10, again confirming statistical significance. Similar to BTC, the market's excess returns have a statistically significant influence on ETH's excess returns. The small p-value leads us to reject the null hypothesis, meaning that the CAPM model significantly explains a portion of ETH's returns, though other factors beyond market movements may still play a role, as suggested by the low R-squared value of 0.107 to be discussed in greater detail.

## R-Squared ( $R^2$ )

The R-squared ( $R^2$ ) is a statistical measure that indicates the proportion of the variance in the dependent variable that is explained by the independent variable(s) in a regression model. In the context of our models, it represents how well the market's excess returns explain the excess returns of BTC and ETH. R-squared tells us how well the regression model fits the data:

- $R^2 = 0$ : Indicates that the model explains none of the variability in the dependent variable.
- $R^2 = 1$ : Indicates that the model explains all of the variability in the dependent variable.

In other words, the higher the R-squared, the better the model explains the variations in the dependent variable. The mathematical formula for calculating  $R^2$  is:

$$R^2 = \frac{ESS}{TSS}$$

Where:

- ESS: Error Sum of Squares (also known as Residual Sum of Squares or Unexplained Sum of Squares). It measures the variation in the dependent variable that is not explained by the model.

$$ESS = \sum (y_i - \hat{y}_i)^2$$

where  $y_i$  is the observed value of the independent variable and  $\hat{y}_i$  is the predicted value from the model.

- TSS: Total Sum of Squares. This measures the total variation in the dependent variable (excess returns) from its mean:

$$TSS = \sum (y_i - \bar{y})^2$$

where  $\bar{y}$  is the mean of the observed values.

In the BTC model, the R-squared value is 0.086, meaning that the market's excess returns explain approximately 8.6% of the variance in BTC's excess returns. This indicates that while the CAPM model is statistically significant (as shown by the F-test), it only explains a small portion of the variability in BTC's returns. The relatively low R-squared suggests that other factors, beyond market risk, may also influence BTC's returns.

In the ETH model, the R-squared value is 0.107, meaning that the market's excess returns explain about 10.7% of the variance in ETH's excess returns. This is slightly higher than BTC, but still indicates that the model explains only a small proportion of the variability in ETH's returns.

This is consistent with the nature of cryptocurrencies, which tend to be driven by factors beyond traditional market movements, such as speculation and market sentiment. Hence, while the market's excess returns play a role in explaining BTC and ETH returns, their impact can be relatively limited.

### Log-Likelihood

The log-likelihood function is derived from the likelihood function, which represents the probability of the observed data given a set of model parameters. For a regression model, the likelihood function assumes that the errors (residuals) are normally distributed. The log-likelihood is simply the natural logarithm of the likelihood function, which simplifies calculations.

The formula for the log-likelihood ( $\ell$ ) of a regression model with normally distributed errors is:

$$\ell = -\left(\frac{n}{2}\right) \log(2\pi) - \left(\frac{n}{2}\right) \log(\sigma^2) - \left(\frac{1}{2\sigma^2}\right) \sum (y_i - \hat{y}_i)^2$$

where:

$n$  is the number of observations.

$\sigma^2$  is the variance of the residuals.

$y_i$  is the observed value.

$\hat{y}_i$  is the predicted value.

$\sum (y_i - \hat{y}_i)^2$  represents the sum of squared residuals (errors).

- Higher log-likelihood values indicate that the model is more likely to have generated the observed data.

- Lower log-likelihood values suggest that the model does not fit the data well.

For the BTC model, the log-likelihood value of  $-1728.4$  indicates the likelihood of the observed BTC excess returns being generated by the model given its parameters. While a negative log-likelihood value is expected in many models, the main takeaway here is that this number reflects how well the model fits the BTC data.

For the ETH model, the log-likelihood is  $-1396.0$ , which represents the likelihood of the observed ETH excess returns being explained by the CAPM model. Again, while the negative value is common in log-likelihood calculations, the key point is that this number reflects the model's fit to the ETH dataset. The log-likelihood will have further relevance later in this assignment when we compare our results with those from a model with no constant.

### **Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC)**

The AIC and the BIC are two commonly used metrics for model selection. Both criteria help evaluate the quality of a statistical model by balancing goodness of fit with model complexity. In the context of the models for BTC and ETH, AIC and BIC allow us to assess whether the CAPM provides an adequate fit of the data while penalising for any unnecessary complexity in the model. Lower values of AIC or BIC indicate a better balance between the complexity of the model and its fit to the data.

The two criteria differ in the penalty they impose for model complexity:

- **AIC:** Focuses more on the goodness of fit, with a moderate penalty for the number of parameters.
- **BIC:** Imposes a stricter penalty for the number of parameters, thus preferring simpler models compared to AIC, especially as the sample size increases.

The mathematical formulae for AIC and BIC are:

$$AIC = -2\ell + 2k$$

$$BIC = -2\ell + k\log(n)$$

Where:

$\ell$  is the log-likelihood of the model

$k$  is the number of parameters

$n$  is the number of observations

The BIC introduces a stronger penalty for additional parameters compared to AIC, especially when the number of observations ( $n$ ) is large. In the context of our models, we observe that for BTC, the AIC is 3461.0 and the BIC is 3469.0. For ETH, we obtain an AIC of 2796.0 and a BIC of 2804.0. Again, similar to Log-likelihood, the AIC and BIC will be examined further later in this assignment when we compare our results to a model that excludes a constant.

## Omnibus Test

The Omnibus Test (D'Agostino (1971) ; Belanger and D'Agostino (1990)) evaluates the joint hypothesis that both the skewness and kurtosis of residuals are zero, which is consistent with the assumptions of normality (FIN30200 class notes). The test checks for both skewness (asymmetry) and kurtosis (peakedness) simultaneously. The test statistic is compared to a chi-squared distribution with 2 degrees of freedom. The mathematical formula for the Omnibus test is:

$$Omnibus = n \times \left( \frac{S^2}{6} + \frac{(K - 3)^2}{24} \right)$$

where  $S$  is skewness and  $K$  is Kurtosis.

For both our models, we obtained p-values very close to zero, indicating that our residuals are likely not normally distributed.

## Skew

Skewness measures the asymmetry of the residuals' distribution. In a perfectly normal distribution, the skewness would be zero, meaning the data is symmetrically distributed around the mean.

Positive skewness: The right tail is longer, indicating more values above the mean.

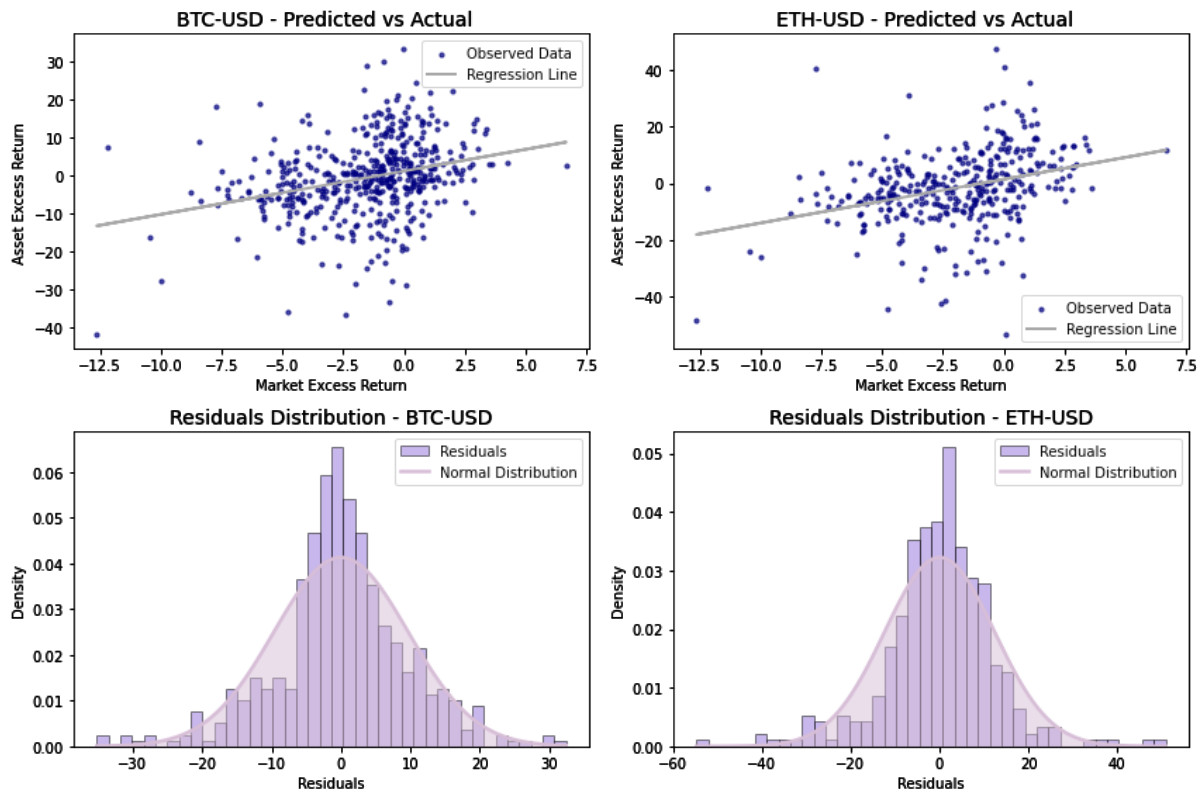
Negative skewness: The left tail is longer, indicating more values below the mean.

For both BTC and ETH, the skewness values are slightly negative, indicating that the residuals are slightly skewed to the left (more values below the mean), however the magnitude is relatively small, so while there is some asymmetry, it is not extreme.

## Kurtosis

Kurtosis measures the tailedness of the distribution of residuals, particularly the presence of outliers. In a normal distribution, kurtosis is 3 (this is sometimes called mesokurtic). Both BTC and ETH exhibit kurtosis values greater than 3, indicating that the residuals have heavy tails (leptokurtic distribution). This suggests that both models have more extreme outliers or deviations than expected under a normal distribution, with ETH having a higher kurtosis value than BTC, indicating even more pronounced outliers.

Figure 2: Classical Linear Regression and Residual Plots



### iii.vi. Normality of Residuals

Now that we commented on various statistical indicators for our models, we would like to pay closer attention to our residuals. The first thing we want to do is check for normality. There are a number of tests we can perform:

#### Visual Inspection

A simple, yet helpful tool to assess the normality of our residuals is plot them on a histogram along with a normal distribution overlay. From initial inspection, it appears that neither our residuals for BTC or ETH are normally distributed however we can conduct a number of additional tests to check the validity of this.

#### Jarque-Bera Test

The Jarque-Bera (JB) test (Jarque and Bera (1987)) is a statistical test used to determine whether the residuals of a regression model are normally distributed. It specifically examines the skewness and kurtosis of the residuals to assess their deviation from a normal distribution. To perform the Jarque-Bera test, we conduct another hypothesis test where:

$H_0$ : The residuals are normally distributed (i.e., skewness = 0 and kurtosis = 3)

$H_0$ : The residuals are not normally distributed.

The Jarque-Bera test uses the skewness and kurtosis of the residuals to calculate a test statistic, which is compared against a chi-squared distribution with 2 degrees of freedom. The mathematical formula for the Jarque-Bera test statistic is:

$$JB = \left(\frac{n}{6}\right) \left(S^2 + \frac{(K - 3)^2}{4}\right)$$

where:

$n$  = the number of observations

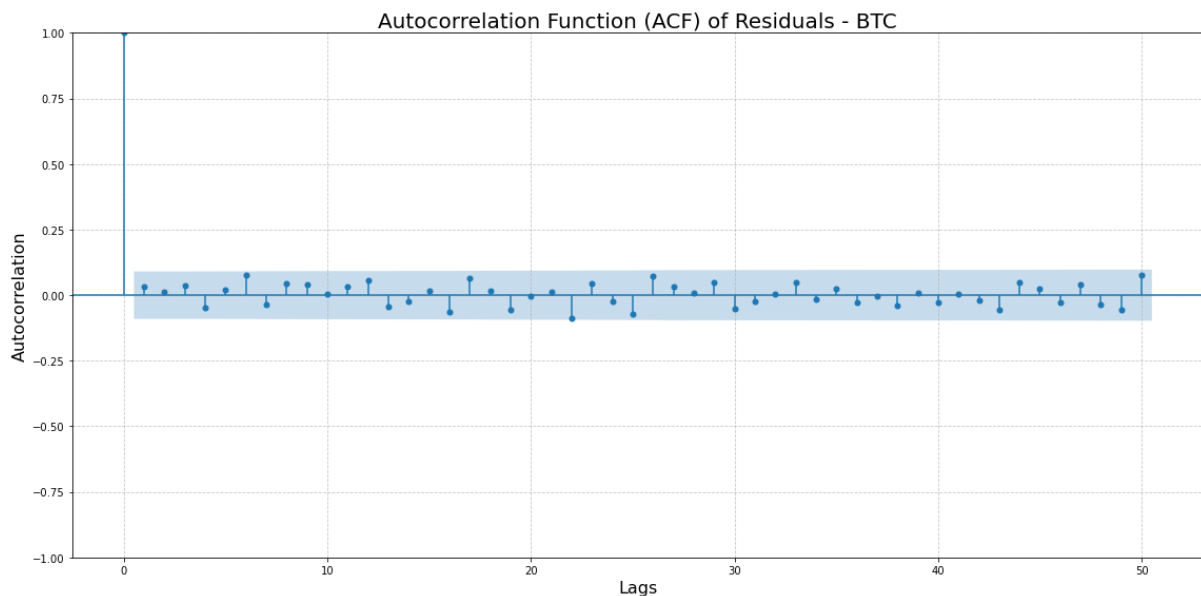
$S$  = the skewness of the residuals

$K$  = the kurtosis of the residuals

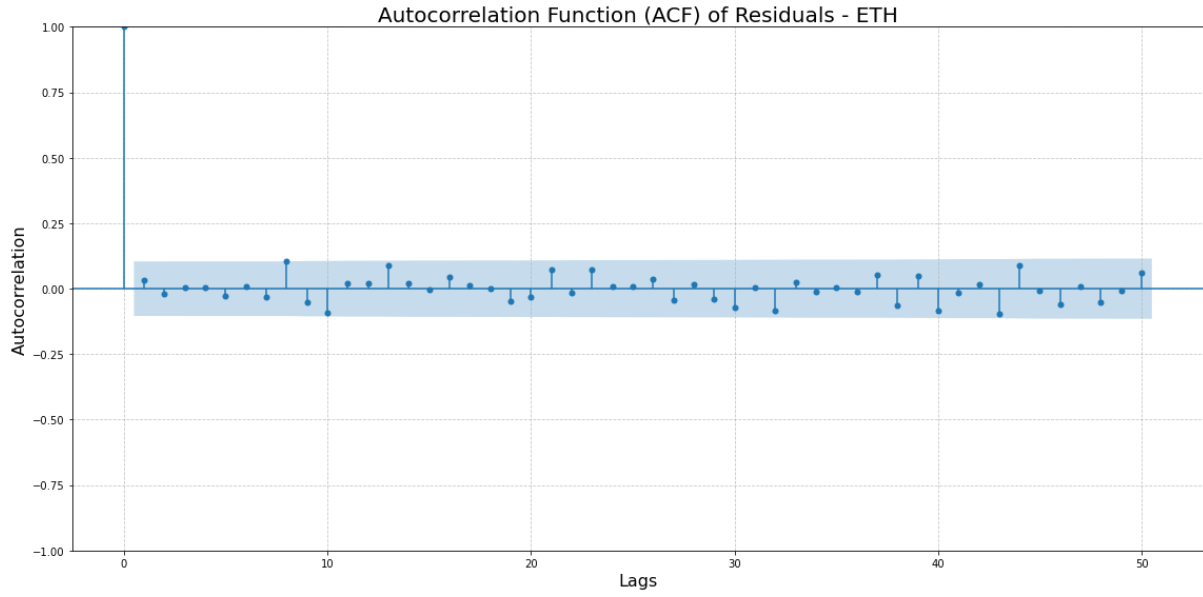
A Jarque-Bera statistic closer to zero indicates a distribution closer to normality. If the p-value of the Jarque-Bera test is less than 0.05, we reject the null hypothesis. Conversely, if the p-value is greater than 0.05, we fail to reject the null hypothesis, meaning the residuals can be considered normally distributed. For both our models, we obtain extremely small p-values, indicating that the residuals for both models likely deviate significantly from normality.

### iii.vii. Testing for Serial Correlation

Figure 3: Autocorrelation Plots for BTC and ETH Residuals







Examining the ACF plots in figure 3, we can see that there are no significant values outside the confidence interval at each lag, suggesting that the residuals are likely independent and thus satisfy the assumption of no serial autocorrelation.

### Durbin-Watson Test

The Durbin-Watson test (Durbin and Watson (1951)) is specifically used to detect the presence of autocorrelation (serial correlation) in the residuals of a regression model. It is particularly focused on first-order correlation, which occurs when the residuals at one time point are correlated with the residuals at the next time point.

$H_0$ : No first-order autocorrelation (residuals are independent)

$H_1$ : Presence of first-order autocorrelation (residuals are serially correlated)

The Durbin-Watson statistic ( $D$ ) is calculated as follows:

$$DW = \frac{\sum_{t=2}^T (\hat{u}_t - \hat{u}_{t-1})^2}{\sum_{t=1}^T \hat{u}_t^2}$$

where:

$\hat{u}_t$  is the residual at time  $t$

$\hat{u}_{t-1}$  is the residual at time  $t - 1$

$T$  is the total number of observations

The Durbin-Watson statistic ( $DW$ ) ranges from 0 to 4:

- $DW \approx 2$  indicates no significant autocorrelation in the residuals
- $DW < 2$  suggests positive autocorrelation in the residuals

- $DW > 2$  suggests negative autocorrelation in the residuals

In addition to the Durbin-Watson statistic, critical values known as the upper ( $d_u$ ) and lower ( $d_l$ ) limits are used to assess the significance of the results. These values depend on the number of observations ( $n$ ) and the number of explanatory variables ( $k$ ) in the model. If the statistic falls between  $d_u$  and  $4 - d_u$  there is no evidence of autocorrelation. If it falls below  $d_l$  or above  $4 - d_l$ , autocorrelation is present. Values between  $d_l$  and  $d_u$  or between  $4 - d_u$  and  $4 - d_l$  indicate inconclusive results. Using Table A2.6 in Brooks (2008), I obtained a lower limit ( $d_l$ ) of 1.52 and an upper limit ( $d_u$ ) of 1.56 at the 1% significance level. Since our Durbin-Watson statistics for BTC (1.934) and ETH (1.930) are greater than the upper bound (1.56), this indicates that there is no evidence of positive autocorrelation in the residuals for either model. Furthermore, both statistics are close to 2, suggesting no significant autocorrelation in the residuals overall.

Before we conclude our analysis for serial correlation, there is one additional test that we can perform:

### Breusch-Godfrey Test

The Breusch-Godfrey test (Breusch (1978) and Godfrey (1978)) is a more general test for higher-order autocorrelation, meaning it can detect correlation at multiple time lags (not just first-order like the Durbin-Watson test). This test is useful when you suspect autocorrelation at lags beyond the immediate past observation. To perform the Breusch-Godfrey test, we complete the following steps:

1. We estimate the original regression model to obtain the residuals ( $e_t$ ):

$$y_t = \beta_0 + \beta_1 x_{1t} + \beta_2 x_{2t} + \dots + \beta_k x_{kt} + e_t$$

where:

$y_t$  is the dependent variable

$x_{1t}, x_{2t}, \dots, x_{kt}$  are the independent variables

$\beta_0, \beta_1, \dots, \beta_k$  are the coefficients to be estimated

$e_t$  is the residual for each observation  $t$

2. We regress the residuals from the original regression on lagged residuals (up to  $p$  lags, where  $p$  is the number of lags we want to test) and the original independent variables:

$$e_t = \alpha_0 + \alpha_1 x_{1t} + \alpha_2 x_{2t} + \dots + \alpha_k x_{kt} + \rho_1 e_{t-1} + \rho_2 e_{t-2} + \dots + \rho_p e_{t-p} + \epsilon_t$$

where:

$e_{t-1}, e_{t-2}, \dots, e_{t-p}$  are the lagged residuals

$x_{1t}, x_{2t}, \dots, x_{kt}$  are the original explanatory variables

$\alpha_0, \alpha_1, \dots, \alpha_k$  are the new coefficients to be estimated for the independent variables

$\rho_1, \rho_2, \dots, \rho_\rho$  are the new coefficients of the lagged residuals

$\epsilon_t$  is the new error term (white noise)

This auxiliary regression checks whether the residuals from the original model are correlated with their past values (i.e., whether or not serial correlation is present).

3. We now compute the Breusch-Godfrey test statistic which is based on the R-squared value from the auxiliary regression of the residuals from Step 2. The test statistic is calculated as follows:

$$LM = nR^2$$

where:

$n$  is the number of observations in the original regression model

$R^2$  is the R-squared from the auxiliary regression (the proportion of the variance in the residuals explained by the independent variables and lagged residuals).

4. We now compare the test statistic to the Chi-Square distribution with degrees of freedom equal to the number of lagged residual terms ( $\rho$ ).

$H_0$ : There is no serial correlation at lags 1 through  $\rho$

$H_0$ : There is serial correlation at one or more of the specified lags

5. If the LM statistic exceeds the critical value from the chi-square distribution at a chosen significance level, we reject the null hypothesis. This suggests that there is significant evidence of serial correlation at the specified lags.

For BTC, we obtained a LM statistic of 7.238 which corresponds to a p-value of 0.7028. The p-value is well above the common significance level of 0.05 which means we fail to reject the null hypothesis that there is no serial correlation in the residuals. In other words, there is no significant evidence of serial correlation at the specified lags in the BTC model. For ETH, we obtained a LM statistic of 9.500 which corresponds to a p-value of 0.4854. Similarly, the p-value is greater than 0.05, meaning we fail to reject the null hypothesis for the ETH model as well, indicating that there is no significant evidence of serial correlation in the residuals at the specified lags.

### iii.viii. Checking for Heteroskedasticity

Now that we have checked our residuals for non-normality and serial correlation, we can perform some final tests to see if the residuals from our models exhibit signs of

heteroskedasticity. One of the assumptions of linear regression is that the variance of the errors is constant, otherwise known as homoscedasticity.

Figure 4: Residuals vs Fitted Values for BTC

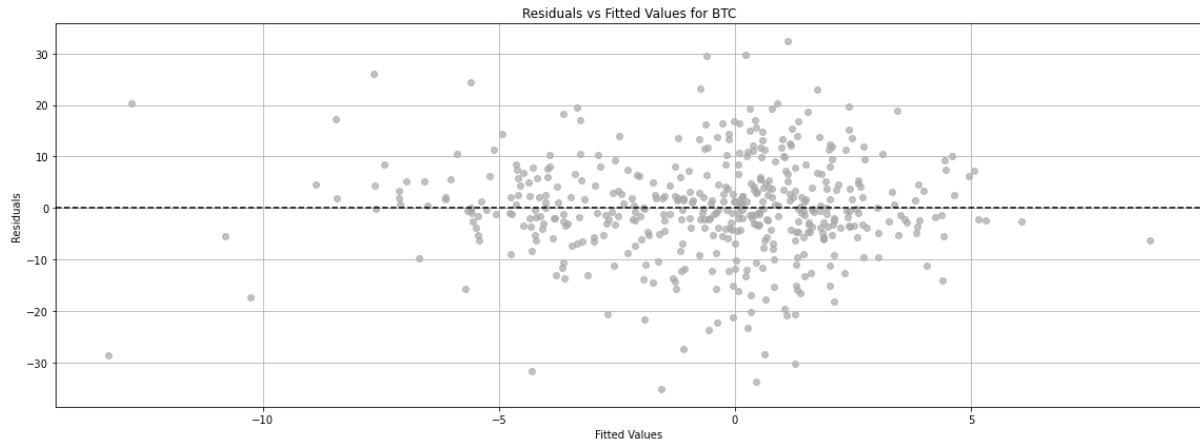
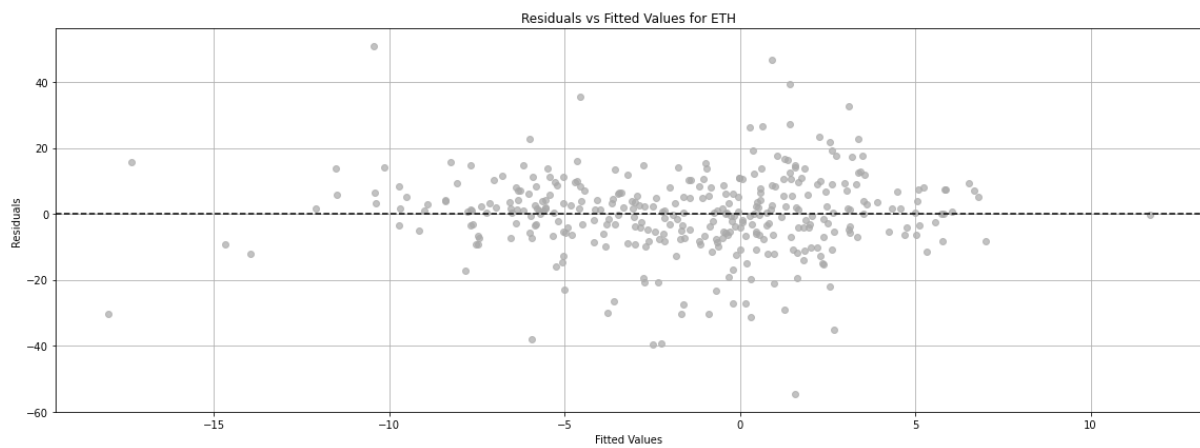


Figure 5: Residuals vs Fitted Values for ETH



## White Test

The White test is used to detect heteroskedasticity in a regression model. The test is flexible because it can detect both pure heteroskedasticity (when the variance of the errors is dependent only on the independent variables) and heteroskedasticity caused by misspecification (such as interaction terms or non-linear relationships).

The steps to perform the white test are as follows:

1. Estimate the original regression model to obtain the residuals ( $e_t$ ):

$$y_t = \beta_0 + \beta_1 x_{1t} + \beta_2 x_{2t} + \cdots + \beta_k x_{kt} + e_t$$

where:

$y_t$  is the dependent variable

$x_{1t}, x_{2t}, \dots, x_{kt}$  are the independent variables

$\beta_0, \beta_1, \dots, \beta_k$  are the coefficients to be estimated

$e_t$  is the residual for each observation  $t$

2. Regress squared residuals ( $e_t^2$ ) on explanatory variables and their squares/interactions. This auxiliary regression captures how the variance of the residuals changes with the independent variables. The auxiliary regression is:

$$e_t^2 = \alpha_0 + \alpha_1 x_{1t} + \alpha_2 x_{2t} + \dots + \alpha_k x_{kt} + \alpha_{k+1} x_{1t}^2 + \dots + \alpha_{k+p} x_{kt}^2 + \text{interaction terms} + \epsilon_t$$

where:

$x_{1t}^2, x_{2t}^2, \dots$  are the independent variables

Interaction terms account for combinations of the independent variables

3. Compute the Test Statistic:

$$LM = nR^2$$

4. Compare the Test Statistic to the Chi-Square Distribution:

The test statistic ( $LM = nR^2$ ) follows a chi-square distribution with degrees of freedom equal to the number of independent variables in the auxiliary regression (including squares and interaction terms).

$H_0$ : The residuals are homoscedastic (i.e., the variance of the residuals is constant across levels of the independent variables)

$H_1$ : The residuals are heteroskedastic (i.e., the variance of the residuals depends on the independent variables or their interactions)

5. Make a Decision:

If the computed LM statistic exceeds the critical value from the chi-square distribution at a chosen significance level (typically  $\alpha = 0.05$ ), we reject the null hypothesis. This suggests that heteroskedasticity is present. If the LM statistic is below the critical value, we fail to reject the null hypothesis, indicating no significant heteroskedasticity.

When we perform the white test on our models, we firstly obtain a Lagrange Multiplier statistic of 2.8169 for BTC which corresponds to a p-value of 0.2445. As the p-value is greater than 0.05, we fail to reject the null hypothesis of homoscedasticity, indicating that there is no significant evidence of heteroskedasticity in our model. The residuals appear to have a constant variance across the levels of the independent variables. Similarly, the F-statistic of 1.4079 has an associated p-value of 0.2457 which is also above 0.05, supporting the conclusion that there is no significant evidence of heteroskedasticity in the model. For

ETH, we obtain a Lagrange Multiplier statistic of 0.3065 which corresponds to a p-value of 0.8579, well above 0.05. Therefore we again fail to reject the null hypothesis and can conclude that there is even stronger evidence in this instance that there is no heteroskedasticity in the model for ETH. The F-statistic of 0.1521 also has a high p-value of 0.8590, reinforcing the conclusion that heteroskedasticity is not evident in the model.

### **Goldfeld-Quandt Test**

Another test we can perform to detect the presence of heteroskedasticity is the Goldfeld-Quandt test (Goldfeld and Quandt (1965)). The test is a useful method to detect heteroskedasticity in a regression model. It works by splitting the data into two subsamples and comparing the variances of the residuals across these subsamples.

Hypotheses:

$H_0$ : The variances of the errors in the two subsamples are equal (i.e.,  $\sigma_1^2 = \sigma_2^2$ )

$H_1$ : The variances of the errors in the two subsamples are not equal (i.e.,  $\sigma_1^2 \neq \sigma_2^2$ )

The test statistic is the ratio of the residual variances from the two subsamples. The larger variance is always placed in the numerator, ensuring that the test statistic is always greater than or equal to 1.

It is computed as:

$$GQ = \frac{s_1^2}{s_2^2}$$

where  $s_1^2$  is the larger of the two sample variances, and  $s_2^2$  is the smaller. A significantly large GQ statistic indicates that heteroskedasticity is present, meaning the variance of the residuals changes across the subsamples.

When we conduct the Goldfeld-Quandt test on our models for BTC and ETH, we obtain F-statistics of 0.6074 and 0.4907 for BTC and ETH respectively, corresponding to p-values of almost 1 in both cases. For both cryptocurrencies, the p-values are very high indicating that we fail to reject the null hypothesis. This suggests that the variances of the residuals in the two subsamples are statistically equal, meaning there is no significant evidence of heteroskedasticity in either model.

### **Breusch-Pagan Test**

The Breusch-Pagan test (Breusch and Pagan (1979)) is an additional test we can perform to detect heteroskedasticity in our models. It is similar to the White test, but typically does not include squares and interaction terms of the independent variables, focusing solely on the linear relationship between the residual variance and the independent variables.

The Breusch-Pagan test statistic is based on the R-squared value from the auxiliary regression of the squared residuals of the independent variables:

$$LM = \frac{nR^2}{2}$$

In both models, BTH and ETH, the Breusch-Pagan test results show that the p-values for the LM statistic and F-statistic are significantly greater than 0.05. This means that for both models, we fail to reject the null hypothesis of homoskedasticity. The residuals exhibit constant variance, and there is no evidence of heteroskedasticity in either model. This is an ideal outcome, as heteroskedasticity can affect the efficiency of regression estimates and the validity of statistical inferences. In the event that we did find evidence of heteroskedasticity, a practical approach would be to re-estimate our models using robust standard errors. Although our models show no evidence of heteroskedasticity, we will still perform this analysis for the purposes of this assignment.

### III. Linear Regression using Robust Standard Errors

#### i. Running our Models with Robust Standard Errors

While our initial diagnostic tests did not reveal signs of heteroskedasticity in the residuals for both BTC and ETH models, we still apply robust standard errors to ensure valid statistical inference. Specifically, we employ Newey-West standard errors, which are heteroskedasticity and autocorrelation consistent (HAC). This allows us to address potential autocorrelation and heteroskedasticity in the residuals that may not have been detected in earlier tests.

Newey-West standard errors, as proposed by Newey and West (1987), are particularly well-suited for time series data, where residuals may exhibit autocorrelation and changing variance over time. In our analysis of weekly log excess cryptocurrency returns, these issues are particularly relevant given the volatility and potential serial correlation of returns over time. Applying Newey-West standard errors allows us to obtain more reliable standard errors, ensuring that our hypothesis testing and confidence intervals remain robust, even if autocorrelation or heteroskedasticity is present.

By applying Newey-West standard errors, we are able to maintain the integrity of our statistical inference and mitigate any risks of bias in the standard errors due to autocorrelation or heteroskedasticity. This step is essential in financial data analysis, where time-dependent patterns are often observed. As a result, we can confidently interpret the coefficients of our CAPM model with the added assurance that our conclusions are not compromised by underlying data characteristics.

When we re-run our models using Newey-West standard errors, we obtain the following results:

Table 5: Simple Linear Regression Results (Robust Standard Errors)

|             | BTC       |           | ETH       |           |
|-------------|-----------|-----------|-----------|-----------|
|             | $\beta_0$ | $\beta_1$ | $\beta_0$ | $\beta_1$ |
| Coefficient | 1.1738    | 1.1436    | 1.4391    | 1.5398    |
| Std. Error  | 0.572     | 0.153     | 0.864     | 0.241     |

Table 6: Significance Results for BTC (Robust Standard Errors)

## Significance Test for BTC

| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| Intercept ( $\beta_0$ )   | 2.051 | 1.96           | 0.040   | [0.052              | 2.296] |
| Coefficient ( $\beta_1$ ) | 7.468 | 1.96           | 0.000   | [0.843              | 1.444] |

We can see that for both models, the standard errors for our coefficients have increased. In our BTC model using robust standard errors, the standard error of our intercept ( $\beta_0$ ) increased from 0.522 to 0.572 while the standard error of our coefficient  $\beta_1$  decreased from 0.172 to 0.153. In our ETH model, the standard error of our intercept ( $\beta_0$ ) increased from 0.806 to 0.864 and the standard error for  $\beta_1$  increased from 0.236 to 0.241. The increase in standard errors for the intercepts in both models and for  $\beta_1$  in ETH suggests that heteroskedasticity may have been present, leading to underestimation of the variability in the original model. However, the decrease in the standard error for  $\beta_1$  in the BTC model indicates that the original model may have overestimated the variability of this coefficient and the robust standard errors correct this by providing a more precise estimate.

BTC:**Intercept ( $\beta_0$ )**

The intercept in the non-robust model was statistically significant with a t-statistic of 2.249, corresponding to a p-value of 0.025. However, after applying robust standard errors, the t-statistic decreased to 2.051, resulting in a p-value of 0.040. The confidence interval also widened slightly from [0.148, 2.200] to [0.052, 2.296], which reinforces the idea that the intercept is now estimated with more caution after the adjustment for potential issues.

**Coefficient of Market Excess Return ( $\beta_1$ ):**

The coefficient estimate remained the same (1.1436) across both models. Interestingly, the robust model showed an increase in the t-statistic, moving from 6.644 to 7.468, indicating stronger significance of the coefficient after adjusting for heteroskedasticity and autocorrelation. The p-value remained highly significant (0.000), and the confidence interval for the robust model narrowed slightly from [0.805, 1.482] to [0.843, 1.444], which signals greater confidence in the precision of the estimate after applying the robust standard errors.



Table 7: Significance Results for ETH (Robust Standard Errors)

| Significance Test for ETH |       |                |         |                     |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
| Intercept ( $\beta_0$ )   | 1.666 | 1.96           | 0.096   | [-0.254             | 3.132] |
| Coefficient ( $\beta_1$ ) | 6.379 | 1.96           | 0.000   | [1.067              | 2.013] |

ETH:

### Intercept ( $\beta_0$ )

In the non-robust model, the intercept had a t-statistic of 1.787 and a p-value of 0.075, which indicated that the intercept was not statistically significant at the 5% level. After applying robust standard errors, the t-statistic decreased to 1.666, and the p-value increased to 0.096, meaning that the intercept is now further from statistical significance at the 5% level. The confidence interval widened slightly, from [-0.145, 3.023] to [-0.254, 3.132], which further reflects the increased uncertainty around the intercept estimate when accounting for potential heteroskedasticity and autocorrelation in the data.

### Coefficient of Market Excess Return ( $\beta_1$ ):

As with BTC, the coefficient of market excess returns in the ETH model remained statistically significant across both models. The t-statistic for the coefficient decreased slightly from 6.518 to 6.379, but it remained highly significant with a p-value of 0.000. The confidence interval for the robust model widened only slightly, from [1.075, 2.004] to [1.067, 2.013], reflecting continued confidence in the precision of this estimate even after applying robust standard errors.

## IV. Constructing a Confidence Interval for $\beta$

In this part of the analysis, we construct a 95% confidence intervals for  $\beta_1$  for both BTC and ETH.  $\beta$  measures the systematic risk of an asset relative to the market. A beta greater than 1 indicates that the asset is more volatile than the market, while a beta less than 1 suggests that the asset is less volatile. Our 95% confidence levels for the two models are:

Table 8: Confidence Intervals for  $\beta$ 

| $\alpha = 0.05$ | $\beta_1$ | Std. Error | Critical z | P> z  | Confidence Interval |        |
|-----------------|-----------|------------|------------|-------|---------------------|--------|
| BTC             | 1.1436    | 0.153      | 0.938      | 0.348 | [0.843              | 1.444] |
| ETH             | 1.5398    | 0.241      | 2.236      | 0.025 | [1.067              | 2.013] |

We can now conduct the following hypothesis test for both models:

$$H_0: \beta = 1$$

$$H_1: \beta \neq 1$$

For BTC, we obtain a 95% confidence interval of [0.843, 1.444]. As the interval contains 1, we fail to reject the null hypothesis that  $\beta = 1$ . Personally, I find it somewhat surprising that we fail to rule out the possibility that  $\beta = 1$ , particularly given Bitcoin's reputation for extreme volatility and speculation. I expected the beta to be significantly greater than 1, reflecting much larger price swings compared to the market. Additionally, I had anticipated that Bitcoin might exhibit returns in excess of the market, due to its perceived risk profile as an asset with high upside potential during periods of strong market growth. The real-world implications of this are that a beta close to 1 suggests that Bitcoin's returns are somewhat in line with the market, with only a slightly amplified reaction to market movements (c.1.14% for every 1% market return). The upper bound of the confidence interval, at 1.444, indicates that Bitcoin can indeed experience more volatile swings, although the baseline is not what I expected.

For ETH, the estimated beta is 1.5398, with a 95% confidence interval of [1.067, 2.013]. As this confidence interval excludes 1, there is sufficient evidence to reject the null hypothesis and accept the alternative hypothesis that  $\beta \neq 1$ . Unlike Bitcoin, this result aligns more closely with my expectations of a higher-risk, speculative asset. The fact that Ethereum's beta is comfortably above 1 and its upper bound exceeds 2, indicates that Ethereum is highly sensitive to market movements, with the potential for both larger gains in bull markets and more severe losses during downturns.

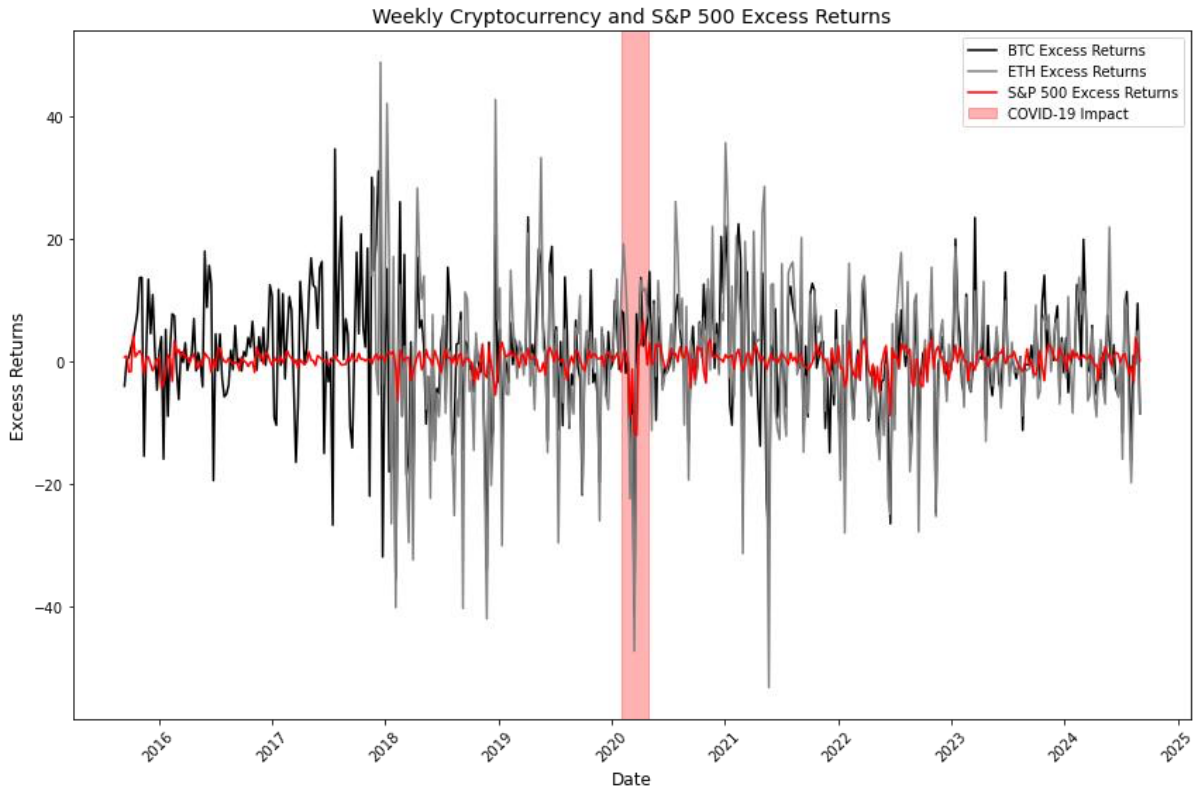
In summary, while ETH's beta aligns with my expectations, I believe that our result for BTC perhaps exploits some limitations of the Capital Asset Pricing Model. It is inarguable that Bitcoin is a highly volatile asset and therefore an investor would naturally expect such risk to be matched with appropriate returns. As such, I believe that our result says less about the returns of BTC but more about the CAPM's inability to appropriately capture extreme levels of volatility. This is something that I shall discuss further in the concluding passage of this assignment.

## V. Splitting our Data in Two

In the next part of this assignment, we will split each of our samples for BTC and ETH into two parts. We will then implement the Chow test to check for the presence of parameter instabilities across both samples.

There are a number of different ways of splitting our data in two, however, in the context of financial datasets, it is often useful to look at periods of extraordinary market volatility. Examples could include global financial crises or significant market events, such as the Covid-19 pandemic. The SP500 recorded its two worst performances in our dataset in the week of and the week following the onset of the pandemic (week ending

Figure 6: Weekly Excess Returns



15<sup>th</sup> March 2020). Similarly, BTC recorded its worst performance of -40.8% while ETH was down -47.2%. As such, it seems like an appropriate timepoint at which to split our data to check for parameter instabilities. To support our decision further and to identify any influential time points that might disproportionately affect the models' results, we can use leverage (hat values). In regression analysis, the leverage (also known as hat values) is a measure used to assess the influence of individual data points on the model's fit. Leverage values help identify outliers or influential observations that can disproportionately affect the regression results.

Leverage values are derived from the hat matrix (H), which is part of the formula used to calculate the predicted values in a regression model. The diagonal elements of this matrix are called hat values or leverages.

The formula for the leverage of a particular observation  $i$  is given by:

$$h_i = X_i(X^T X)^{-1} X_i^T$$

where:

$X$  is the matrix of independent variables (predictors)

$h_i$  is the leverage value for observation  $i$

The matrix  $X^T X$  represents the covariance matrix of the predictors and  $(X^T X)^{-1}$  is its inverse.

### Range of Leverage:

Leverage values range from 0 to 1.

Points with higher leverage values (usually much higher than the average leverage, which is  $\frac{p}{n}$ , where  $p$  is the number of predictors and  $n$  is the number of observations) indicate that the observation has a stronger influence on the model.

### Identifying High Leverage Points:

Typically, if  $h_i$  is significantly higher than  $2\frac{p}{n}$ , the point is considered a high-leverage point.

High-leverage points do not necessarily indicate outliers, but when combined with large residuals, they can skew the model significantly.

As demonstrated in figure 7 and 8, the 15<sup>th</sup> March 2020 is identified as being the most influential timepoint in our models for both BTC and ETH, further reinforcing this date as a key period in the data.

Figure 7: Leverage (BTC)

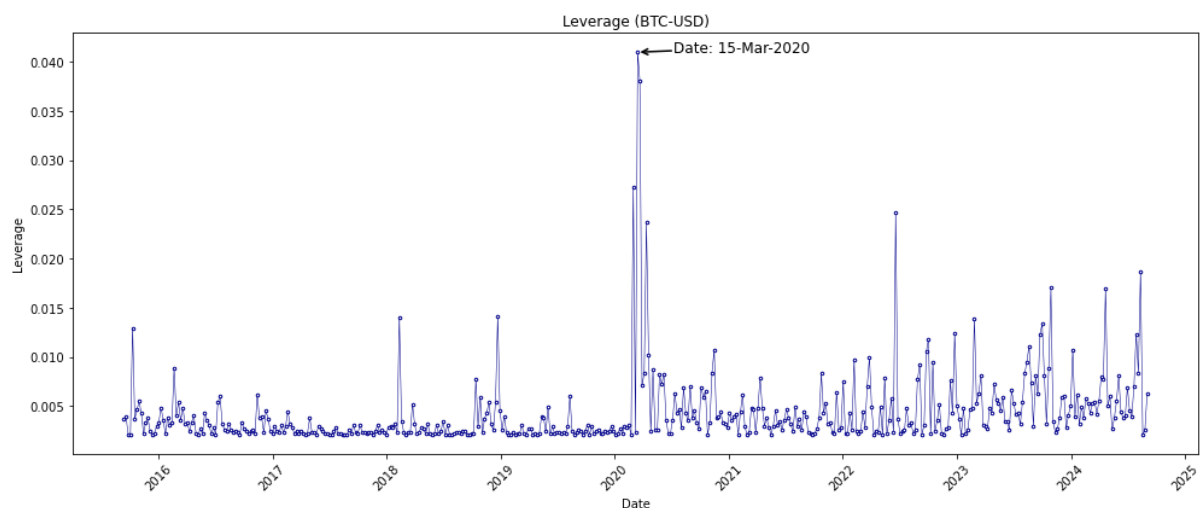
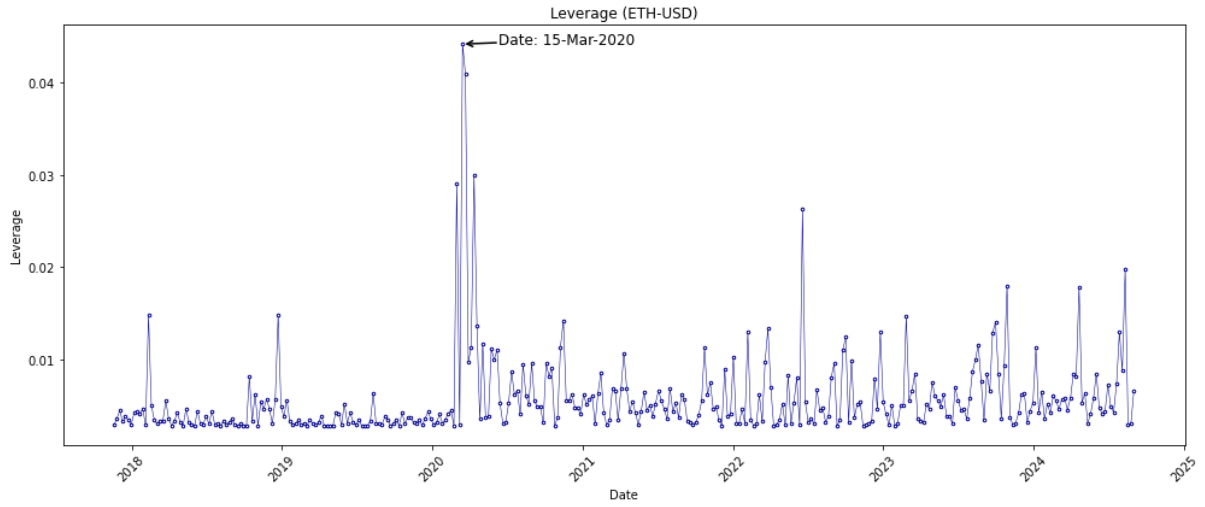


Figure 8: Leverage (ETH)



## i. Running Regression on BTC Subsets

Now that we have identified this date as an optimal split point, we can create two subsamples and run regression on each. We obtain the following results:

Table 9: Simple Linear Regression Results (BTC)

|                    | BTC Subsample 1 |           | BTC Subsample2 |           |
|--------------------|-----------------|-----------|----------------|-----------|
|                    | $\beta_0$       | $\beta_1$ | $\beta_0$      | $\beta_1$ |
| Coefficient        | 1.4094          | 1.0952    | 0.8547         | 1.1292    |
| Std. Error         | 0.833           | 0.414     | 0.658          | 0.174     |
| R-Squared          | 0.029           |           | 0.154          |           |
| Adjusted R-Squared | 0.025           |           | 0.151          |           |
| F-Statistic        | 7.003           |           | 42.32          |           |
| Prob (F-Statistic) | 0.00869         |           | 4.72e-10       |           |
| Log-Likelihood     | -891.28         |           | -829.87        |           |
| AIC                | 1787.           |           | 1664.          |           |
| BIC                | 1793.           |           | 1671.          |           |
| Omnibus            | 8.808           |           | 11.862         |           |
| Prob (Omnibus)     | 0.012           |           | 0.003          |           |
| Skew               | -0.216          |           | -0.264         |           |
| Kurtosis           | 4.082           |           | 4.349          |           |
| Durbin-Watson      | 1.965           |           | 1.848          |           |
| Jarque-Bera (JB)   | 13.286          |           | 20.443         |           |
| Prob (JB)          | 0.00130         |           | 3.64e-05       |           |

## i.i. Statistical Inference (BTC)

Table 10: Significance Results for BTC Subsample 1

| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| Intercept ( $\beta_0$ )   | 1.692 | 1.96           | 0.092   | [-0.232             | 3.051] |
| Coefficient ( $\beta_1$ ) | 2.646 | 1.96           | 0.009   | [0.280              | 1.911] |

Table 11: Significance Results for BTC Subsample 2

| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| Intercept ( $\beta_0$ )   | 1.299 | 1.96           | 0.195   | [-0.441             | 2.151] |
| Coefficient ( $\beta_1$ ) | 6.505 | 1.96           | 0.000   | [0.787              | 1.471] |

### Coefficient Interpretation and Significance

For subsample 1, we obtain an intercept. ( $\beta_0$ ) of 1.4094 however the p-value of 0.092 indicates that this coefficient is not statistically significant at the 5% level. The confidence interval contains zero, further confirming that the intercept is not statistically significant. The coefficient ( $\beta_1$ ) of 1.0952 indicates that for every 1% increase in the excess return of the market, Bitcoin's excess returns increase by 1.0952% on average. The p-value of 0.009 means that the coefficient is statistically significant at the 1% level, indicating that there is strong evidence of a relationship between Bitcoin's returns and the market's excess returns in this period. Furthermore the confidence interval does not contain zero, reaffirming the significance of this relationship.

For subsample 2, we obtain an intercept ( $\beta_0$ ) of 0.8547 and its p-value of 0.195 suggests that this result is not statistically significant. Our estimate for  $\beta_1$  of 1.1292 has a very low p-value close to 0 suggesting that it is again statistically significant at the 1% level, as in subsample 1. We fail to reject the null hypothesis that  $\beta_1 = 1$  in both samples, which I again find quite surprising.

### R-Squared

For subsample 1, a very low  $R^2$  value of 0.029 indicates that only 2.9% of the variation in BTC's excess return is explained by the market's excess return in this period. This indicates a relatively weak relationship between the two in subsample 1 and a very poor fit.

In subsample 2, we obtain a higher  $R^2$  of 0.154, which reflects a stronger relationship than in subsample 1. While still not a high  $R^2$ , the increase between the two subsamples indicates that BTC's returns might have become more closely linked to the market post-pandemic.

## **F-Statistic**

In subsample 1, the F-statistic is 7.003, corresponding to a p-value of 0.00869, indicating that the model is statistically significant at the 1% level. This suggests that the market's excess returns do explain some variation in Bitcoin's excess returns, though the relatively low R-squared indicates that the explanatory power is limited in this period.

For subsample 2, the F-statistic is much higher at 42.32, with an extremely small p-value of  $4.72e-10$ , indicating a highly significant model. This reflects a much stronger relationship between the market's excess returns and Bitcoin's returns in this period, reinforcing the idea that Bitcoin's returns became more closely tied to market movements over time.

## **Durbin-Watson**

In subsample 1, the DW statistic is 1.965, which is very close to 2, indicating that there is little to no autocorrelation in the residuals for this period. This suggests that the model's residuals are not systematically related to each other, which is a good sign of model validity.

In subsample 2, we obtain a DW statistic of 1.848, which is slightly below 2 but still close enough to indicate that there is no strong evidence of autocorrelation. While it is slightly lower than the value in subsample 1, it still suggests that the residuals are not significantly autocorrelated, meaning the model's assumptions regarding the independence of errors hold well for this period too.

## **Jarque-Bera**

In subsample 1 for BTC, the Jarque-Bera statistic is 13.286, with a p-value of 0.0013, indicating that the residuals do not follow a normal distribution.

In subsample 2, the Jarque-Bera statistic is even higher, at 20.443, with a very small p-value of  $3.64e-05$ . This also indicates that the residuals are not normally distributed, and the deviation from normality is even more pronounced in this subsample. The non-normality of the residuals could suggest the presence of outliers or other factors influencing Bitcoin's returns during this period.

## **i.ii. The Chow Test**

The Chow Test is a statistical test used to determine whether there is a structural break between two different subsamples of a dataset. Specifically, it tests whether the coefficients of a regression model differ significantly between two periods or groups, indicating that the relationship between the independent and dependent variables has changed over time. The Chow Test compares the residual sum of squares of three models:

1. The full model
2. A subsample 1 model
3. A subsample 2 model

We test the following hypothesis:

$H_0$ : The parameters are stable over time

$H_1$ : There is a structural break in the data

The test checks whether the coefficients from the two subsample models differ significantly from each other. If they do, it suggests that the relationship between the variables has changed (i.e., a structural break). For the Chow Test, a test statistic is calculated and then compared to a critical value from the F-distribution, which is an  $F(k, T - 2k)$ . If the test statistic is greater than the critical value, we reject the null hypothesis that the parameters are stable over time. The mathematical formula for calculating the Chow Test statistic is (Brooks, 2008):

$$\text{test statistic} = \frac{RSS - (RSS_1 + RSS_2)}{RSS_1 + RSS_2} \times \frac{T - 2k}{k}$$

where:

$RSS$  = the residual sum of squares for the whole sample

$RSS_1$  = the residual sum of squares for subsample 1

$RSS_2$  = the residual sum of squares for subsample 2

$T$  = number of observations

$2k$  = the number of regressors in the ‘unrestricted’ regression (as it comes in two parts)

$k$  = the number of regressors in (each) ‘unrestricted’ regression

The Chow Test results for BTC indicate that there is no significant structural break in the relationship between Bitcoin's excess returns and market excess returns, despite splitting the data at the onset of the COVID-19 pandemic. The F-statistic of 0.219 and p-value of 0.803 suggest that the pandemic did not lead to a substantial change in Bitcoin's behaviour relative to the market. I am slightly surprised, as I would have expected some instability as a consequence of the extreme levels of volatility seen during the week ending 15<sup>th</sup> March 2020. One possible reason for this, however, may be due to the fact that our data is weekly, rather than daily. As such, the more granular, sudden volatility spikes may have been smoothed out to some extent.

Given our results in the Chow Test, we can perform one additional test to search for structural break points in our models.



## Quandt-Andrews Test

Unlike the Chow test, which requires a pre-specified breakpoint, the Quandt-Andrews test (Quandt (1960)) searches for breakpoints across the data, identifying points where the relationship between the dependent and independent variables changes.

$H_0$ : There is no structural break at any point

$H_1$ : There is a structural break at some point

- The test essentially performs a Chow test at each potential breakpoint and calculates an F-statistic for each.
- The breakpoints are evaluated over a certain percentage of the data, typically leaving out the first and last 15% of the observations (i.e., trimming the ends).
- The test outputs the maximum F-statistic, which corresponds to the most likely breakpoint, along with the associated p-value.

The F-statistic for the Quandt-Andrews test is computed as follows:

$$F = \frac{\frac{(RSS_R - RSS_U)}{m}}{RSS_U / (n - 2k)}$$

where:

$RSS_R$  and  $RSS_U$  are the residual sum of squares of the restricted (pooled) and unrestricted models (subsamples split at the breakpoint)

$m$  is the number of restrictions (parameters tested)

$n$  is the number of observations

$k$  is the number of parameters in the unrestricted model

For BTC, we observe a test statistic of 1.3051, which corresponds to a p-value of 0.0663. This result is not statistically significant at the 5% level, therefore indicating no structural break. For ETH, we obtain a test statistic of 0.6863 with a p-value of 0.7339, again suggesting no structural break. This set of results further reinforces our conclusions drawn from the Chow test, that there do not appear to be any structural breakpoints in our data and therefore our parameters are stable. It is worth noting, however, that our result for BTC is statistically significant at the 10% level, indicating that there may well indeed be a structural break in the model despite our findings.

## ii. Running Regression on ETH Subsets

Performing the same steps for ETH as we did for BTC, we obtain the following regression results:

Table 12: Simple Linear Regression Results (ETH)

|                    | ETH Subsample 1 |           | ETH Subsample2 |           |
|--------------------|-----------------|-----------|----------------|-----------|
|                    | $\beta_0$       | $\beta_1$ | $\beta_0$      | $\beta_1$ |
| Coefficient        | 1.0555          | 1.9789    | 1.8930         | 1.4524    |
| Std. Error         | 1.917           | 0.696     | 0.809          | 0.218     |
| R-Squared          | 0.063           |           | 0.161          |           |
| Adjusted R-Squared | 0.055           |           | 0.157          |           |
| F-Statistic        | 8.083           |           | 44.24          |           |
| Prob (F-Statistic) | 0.00525         |           | 2.08e-10       |           |
| Log-Likelihood     | -506.83         |           | -874.13        |           |
| AIC                | 1018.           |           | 1752.          |           |
| BIC                | 1023.           |           | 1759.          |           |
| Omnibus            | 8.518           |           | 40.432         |           |
| Prob (Omnibus)     | 0.014           |           | 0.000          |           |
| Skew               | 0.175           |           | -0.669         |           |
| Kurtosis           | 4.744           |           | 6.508          |           |
| Durbin-Watson      | 1.876           |           | 1.936          |           |
| Jarque-Bera (JB)   | 16.087          |           | 136.830        |           |
| Prob (JB)          | 0.000321        |           | 1.94e-30       |           |

## ii.i Statistical Inference

Table 13: Significance Results for ETH Subsample 1

| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| Intercept ( $\beta_0$ )   | 0.551 | 1.96           | 0.583   | [-2.741             | 4.852] |
| Coefficient ( $\beta_1$ ) | 2.843 | 1.96           | 0.005   | [0.601              | 3.357] |

Table 14: Significance Results for ETH Subsample 2

| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| Intercept ( $\beta_0$ )   | 2.339 | 1.96           | 0.020   | [0.298              | 3.488] |
| Coefficient ( $\beta_1$ ) | 6.651 | 1.96           | 0.000   | [1.022              | 1.883] |

### **Coefficient Interpretation and Significance:**

In subsample 1, we obtain an intercept ( $\beta_0$ ) of 1.0555 and a slope coefficient ( $\beta_1$ ) of 1.9789. The p-value of our intercept is 0.583, so it is not statistically significant while our estimate for  $\beta_1$  is statistically significant at the 1% level. For subsample 2, we obtain estimates of 1.8930 and 1.4524 for  $\beta_0$  and  $\beta_1$  respectively. Both estimates are statistically significant at the 5% level, while a very low p-value of close to zero for  $\beta_1$  indicates that it is also statistically significant at the 1% level. For subsample 1, a 1% increase in the market's excess returns would lead to an average increase in Ethereum's excess returns of 1.9789% pre-Covid. In contrast, for every 1% increase in the market's excess returns post-Covid, Ethereum's excess returns increase by 1.4524% on average. While the estimates for  $\beta_1$  across subsample 1 and subsample 2 are both well above 1, we can see that our estimate for subsample 1 of 1.9789 is notably higher. While this could signify that Ethereum tended to significantly outperform the market pre-Covid, it could also be due to limitations in our dataset, as subsample 2 is significantly larger than subsample 1, meaning there is more data to go off and as such, a greater likelihood of additional variation in weekly returns.

### **R-Squared**

In subsample 1, the  $R^2$  value is 0.063, meaning that only 6.3% of the variation in ETH's excess returns is explained by the market's excess returns. This low value suggests that during this period, Ethereum's returns were primarily driven by factors other than the traditional market.

In subsample 2, the  $R^2$  increases to 0.161, indicating that 16.1% of the variation in Ethereum's returns is explained by the market. While still modest, this higher value suggests that Ethereum became more closely linked to market movements in subsample 2. Overall, the increase in R-squared between the two periods reflects a stronger relationship between Ethereum and the market in the second subsample, though much of Ethereum's returns remain unexplained by the market in both periods.

### **F-Statistic**

In subsample 1, the F-statistic is 8.083 with a corresponding p-value of 0.00525, indicating that the model is statistically significant at the 1% level. This means that the market's excess returns do explain some variation in Ethereum's returns during this period, though the relationship is relatively weak, as seen in the low  $R^2$  value.

In subsample 2, the F-statistic is much higher at 44.24 with a p-value of 2.08e-10, indicating a much stronger and highly significant relationship between the market's excess returns and Ethereum's returns. The higher F-statistic in subsample 2 reflects a greater explanatory power of the market in this period, consistent with the higher  $R^2$  value. Overall, the F-statistics show that the relationship between Ethereum's returns and the market is significant in both periods, with a stronger connection in subsample 2.

### **Durbin-Watson**

In subsample 1, the DW statistic is 1.876, which is close to 2, indicating little to no autocorrelation in the residuals for this period. This suggests that the model's residuals are independent, supporting the validity of the model.

In subsample 2, we obtain a DW statistic of 1.936, again close to 2, similarly indicating no significant autocorrelation in the residuals. The near 2 values in both subsamples suggest that the residuals are not systematically related, meaning the assumption of independent errors likely holds for both models.

### Jarque-Bera

In subsample 1, we obtain a Jarque-Bera statistic of 16.087, which corresponds to a p-value of 0.000321, indicating that the residuals are not normally distributed. This suggests that the model's assumptions about normally distributed errors may be violated, and there could be the presence of outliers or non-normal behaviour affecting Ethereum's returns in this period.

In subsample 2, the Jarque-Bera statistic is even higher at 136.830 with a very small p-value of  $1.94e-30$ , again showing that the residuals are likely not normally distributed, and the deviation from normality is even more pronounced in this period.

Overall, the non-normality of the residuals in both subsamples suggests that caution should be exercised when interpreting the model's results, as violations of the normality assumption can impact statistical inference.

### ii.ii. The Chow Test

The Chow Test results for ETH show an F-statistic of 1.251 and a corresponding p-value of 0.287, indicating that there is no significant structural break in the relationship between Ethereum's excess returns and the market's excess returns across the two subsamples. Since the p-value is well above the 0.05 significance level, we fail to reject the null hypothesis, meaning that the relationship between Ethereum and the market remained relatively stable despite the split point.

## VI. Linear Regression Without A Constant

The final part of this assignment is to perform linear regression on BTC and ETH again, however this time excluding the constant term. The formula for our new regression model without a constant can be expressed as:

$$y = \beta_1 x + \epsilon$$

where:

$y$ : Dependent variable ( $R_{it} - R_{ft}$ )

$x$ : Independent variable ( $R_{mt} - R_{ft}$ )

$\beta_0$ : Intercept

$\beta_1$ : Slope

$\epsilon$ : Error term (or residual)

We can expect the exclusion of the constant to have a number of implications on our model. Firstly, our regression model will now pass through the origin, meaning that when the weekly excess return of the market is zero, the weekly excess return of the cryptocurrency will also be zero. It is also worth noting that one of the assumptions of the Capital Asset Pricing Model is that the constant term is zero anyway, so it will be quite interesting to see how our models without a constant compare to those we ran at the beginning of this paper. It is also worth noting that for our original Ethereum model, we failed to reject the null hypothesis that  $\beta_0 = 0$  at the 5% level of significance.

Furthermore, by excluding the constant, the  $R^2$  value could be negative, implying that the sample average,  $\bar{y}$ , explains more of the variation in  $y$  than the explanatory variables. Secondly, a regression with no intercept parameter could lead to biases in the slope coefficient estimates (Brooks, 2008).

Additional effects we could expect to witness include the possibility of changes to our estimates for  $\beta_1$ , the coefficient of the excess market return. We may observe larger or smaller coefficients depending on the data and how much of the variation was captured by the constant.

When we run our regression models without a constant we obtain the following results:

Table 15: Simple Linear Regression Results (No Intercept)

|                                 | BTC       | ETH       |
|---------------------------------|-----------|-----------|
|                                 | $\beta_1$ | $\beta_1$ |
| Coefficient                     | 0.9428    | 1.2958    |
| Std. Error                      | 0.148     | 0.193     |
| R-Squared (Uncentered)          | 0.080     | 0.113     |
| Adjusted R-Squared (Uncentered) | 0.078     | 0.110     |
| F-Statistic                     | 40.70     | 44.92     |
| Prob (F-Statistic)              | 4.27e-10  | 8.11e-11  |
| Log-Likelihood                  | -1730.9   | -1397.6   |
| AIC                             | 3464.     | 2797.     |
| BIC                             | 3468.     | 2801.     |
| Omnibus                         | 20.726    | 32.833    |
| Prob (Omnibus)                  | 0.000     | 0.000     |
| Skew                            | -0.224    | -0.263    |
| Kurtosis                        | 4.411     | 5.849     |
| Durbin-Watson                   | 1.904     | 1.908     |
| Jarque-Bera (JB)                | 42.841    | 124.182   |
| Prob (JB)                       | 4.98e-10  | 1.08e-27  |

Table 16: Significance Results for BTC

## Significance Test for BTC

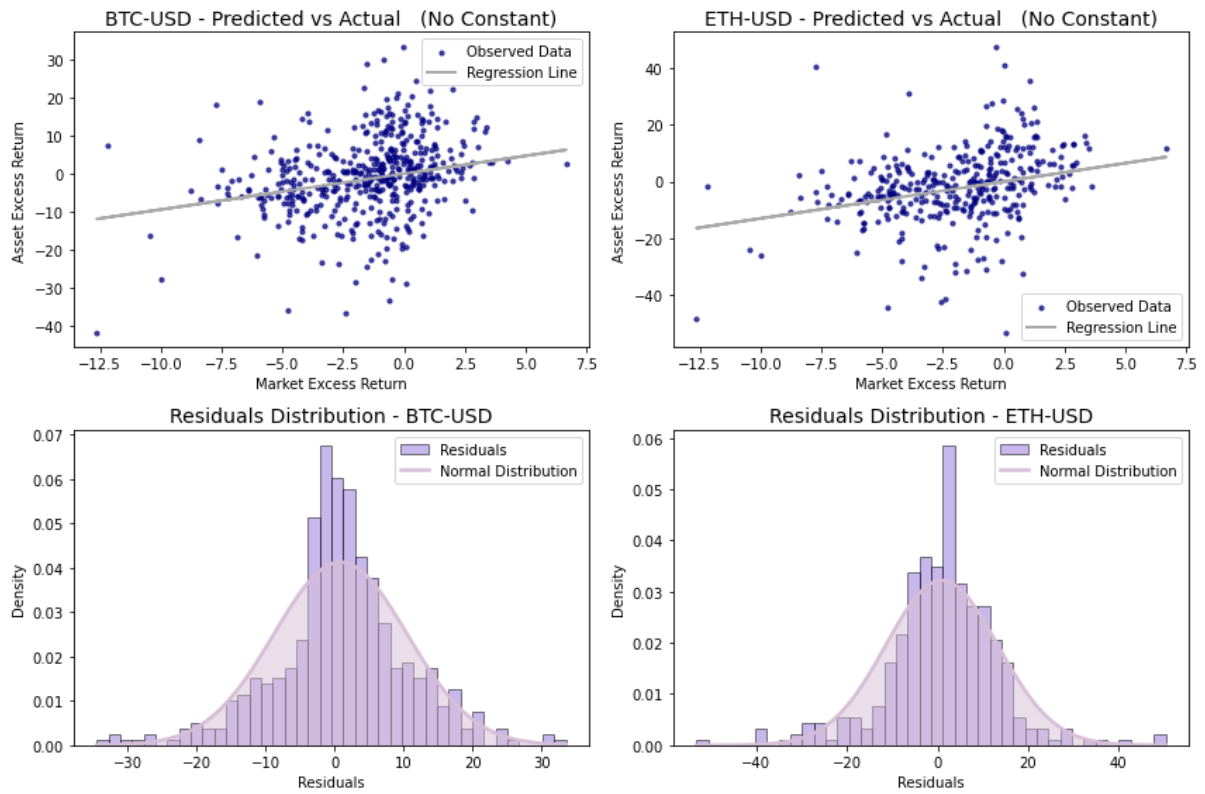
| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| Coefficient ( $\beta_1$ ) | 6.380 | 1.96           | 0.000   | [0.652              | 1.233] |

Table 17: Significance Results for ETH

## Significance Test for ETH

| $\alpha = 0.05$           | t     | Critical Value | p-value | Confidence Interval |        |
|---------------------------|-------|----------------|---------|---------------------|--------|
| Coefficient ( $\beta_1$ ) | 6.702 | 1.96           | 0.000   | [0.916              | 1.676] |

Figure 9: Linear Regression and Residual Plots (No Constant)



## i. Interpretation of Models

For BTC, we obtain an estimate of 0.9428 for  $\beta_1$ , meaning that for every 1% increase in the weekly market excess return, the weekly excess return of Bitcoin is expected to increase by only 0.9428%. We can also see in table 14 that the p-value is effectively zero, meaning this result is statistically significant at the 1% level.

For ETH, we obtain a  $\beta_1$  of 1.2958, which is also statistically significant at the 1% level. In both instances, our estimates for  $\beta_1$  have reduced with the introduction of a constant.

## ii. Further Statistical Inference

### R-Squared

Since the mean value of the dependent variable will not be equal to the mean of the fitted values from the model, the value for R-squared is practically meaningless (Brooks, 2008). Therefore, we will not comment on R-squared as a metric for statistical inference.

### F-Statistic

For BTC, we obtain a F-statistic of 40.70, which corresponds to a p-value of 4.27e-10. For ETH, we observe a F-statistic of 44.92 and a p-value of 8.11e-11. Both results are statistically significant at the 1% level. Our interpretation of the F-statistic changes slightly in this situation however, as we are no longer testing the overall significance of a model but rather the explanatory power of the independent variable over the dependent variable.

### Log-likelihood

For BTC, we have a log-likelihood of -1730.9 and for Ethereum a log-likelihood of -1397.6. To compare these results to those we obtained in our original models with a constant, we can use the Likelihood Ratio (LR) test (Brooks, 2008). The mathematical formula for the LR test is:

$$LR = -2(L_r - L_u)$$

where  $L_r$  is the log-likelihood of our restricted model (without an intercept) and  $L_u$  is the log-likelihood of our unrestricted model (with an intercept). The LR test statistic follows a Chi-squared distribution with degrees of freedom equal to the number of restrictions  $m$ . If the LR statistic is large enough, we reject the null hypothesis and conclude that the unrestricted model fits better.

We obtain a test statistic of 5.0 for BTC which is greater than the critical value of 3.841 for a Chi-squared distribution with 1 degree of freedom, therefore we can conclude that the unrestricted model with the intercept provides a better fit for the data. For ETH, however, we fail to reject the null hypothesis such that removing the intercept does not significantly worsen the model's fit.

### Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC)

For BTC, the unrestricted model (with an intercept) has an AIC of 3461 and a BIC of 3469, while the restricted model (without intercept) has an AIC of 3464 and a BIC of 3468. The slightly lower AIC for the unrestricted model suggests a marginally better fit when including the intercept. For ETH, the unrestricted model has an AIC of 2796 and a BIC of 2804, while the restricted model has an AIC of 2797 and a BIC of 2801. In this case, AIC favours the unrestricted model, while BIC favours the restricted model, which is understandable given that BIC penalises for the number of parameters more heavily than AIC.

### **Omnibus**

For BTC, the Omnibus test statistic is 20.726 with a p-value of 0.000, and for ETH, it is 32.833 with a p-value of 0.000. Compared to the original models, (BTC: 19.837, ETH: 33.407), both models show similar results, with slightly higher test statistics in the no-intercept models. This indicates that the residuals in both cases deviate significantly from normality, whether or not the intercept is included, but the differences between the two models are minimal.

### **Skew**

For BTC, the skew is -0.224 and for ETH, it is -0.263. Compared to the models with the intercept (BTC: -0.211, ETH: -0.264), the skew remains very similar, showing a slight leftward skew in both cases. The removal of the intercept doesn't significantly affect the skew of the residuals, as both models still show mild left-skewness, suggesting a small imbalance with more positive residuals.

### **Kurtosis**

For BTC, the kurtosis is 4.411, and for ETH, it is 5.849. These values are close to those of our original models (BTC: 4.380, ETH: 5.905), indicating that both models display leptokurtic behaviour. The heavy tails suggest the presence of outliers in both models (with and without a constant) and the inclusion or exclusion of the intercept does not significantly alter this characteristic.

### **Durbin-Watson**

For both BTC (DW = 1.904 without intercept, 1.934 with intercept) and ETH (DW = 1.908 without intercept, 1.930 with intercept), the values are close to 2, indicating no significant autocorrelation in the residuals, regardless of whether the intercept is included or not.

### **Jarque-Bera**

For both BTC (JB = 42.841 without intercept, 40.723 with intercept) and ETH (JB = 124.182 without intercept, 128.966 with intercept), the low p-values suggest that the residuals are not normally distributed, and this issue persists whether or not the intercept is included. Thus, neither autocorrelation nor normality of residuals is significantly impacted by the inclusion of the intercept in the models.



## VII. Conclusion

The CAPM, though traditionally effective for understanding the dynamics between traditional assets and the market, shows significant limitations when applied to cryptocurrencies. It inarguably provides a simple and efficient framework to explore the relationship between the excess returns of traditional assets and broader market excess returns. However, I believe CAPM struggles to capture the unique characteristics of cryptocurrencies, such as their extreme volatility and inherent risk, which differ significantly from those of traditional assets. Cryptocurrencies, as speculative and highly volatile assets, exhibit behaviours that often go beyond the explanatory power of a single-factor model like the CAPM thus limiting the model's ability to truly capture the full return of these assets.

There are also some notable limitations to the CAPM when modelling cryptocurrencies. Firstly, the assumption of normality in returns is often violated, as demonstrated in our residual analysis, which revealed non-normality through tests like the Jarque-Bera and Omnibus. Secondly, the assumption that the intercept ( $\beta_0$ ) should be zero often does not hold true for cryptocurrencies. In our original regression models, we found  $\beta_0$  to be statistically significant for Bitcoin, suggesting that cryptocurrencies may exhibit returns independent of market movements, which the CAPM fails to account for. Lastly, CAPM's reliance on the traditional indices as a market proxy might not be ideal for cryptocurrencies. In this study, we used the S&P500 Index however a better approach could involve using a cryptocurrency index or a market representation more closely tied to the crypto sector, which could enhance the model's ability to explain crypto returns.

In conclusion, while CAPM provides a useful starting point for understanding cryptocurrency dynamics, it falls short of fully capturing the complexity of these assets. Alternative models, such as the three-factor pricing model (Shen, Urquhart, & Wang, 2020), which incorporates additional factors and has been shown to outperform CAPM in capturing cryptocurrency behaviour, offer more robust explanations. Likewise, models like the Fama-French three-factor model and the Carhart four-factor model (Brooks, 2008) may offer more comprehensive frameworks that account for additional risk factors, better reflecting the volatility, market dynamics, and idiosyncratic risks inherent to cryptocurrencies.

# References

Wooldridge, J.M. (2010) *Econometric analysis of cross section and panel data*. Cambridge, Massachusetts ; London, England: The MIT Press.

Brooks, C. (2008) *Introductory Econometrics for Finance (3<sup>rd</sup> Edition)*. Cambridge UK: Cambridge University Press

D'Agostino, R.B. (1971) 'An omnibus test of normality for moderate and large size samples', *Biometrika*, 58(2), p. 341. doi:10.2307/2334522.

D'Agostino, R.B., Belanger, A. and D'Agostino, R.B. (1990) 'A suggestion for using powerful and informative tests of normality', *The American Statistician*, 44(4), p. 316. doi:10.2307/2684359.

Jarque, C.M. and Bera, A.K. (1987) 'A test for normality of observations and regression residuals', *International Statistical Review / Revue Internationale de Statistique*, 55(2), p. 163. doi:10.2307/1403192.

DURBIN, J. and WATSON, G.S. (1951) 'Testing for serial correlation in least squares regression. II', *Biometrika*, 38(1–2), pp. 159–178. doi:10.1093/biomet/38.1-2.159.

Breusch, T.S. (1978) 'Testing for autocorrelation in dynamic linear models\*', *Australian Economic Papers*, 17(31), pp. 334–355. doi:10.1111/j.1467-8454.1978.tb00635.x.

Godfrey, L.G. (1978a) 'Testing against general autoregressive and moving average error models when the regressors include lagged dependent variables', *Econometrica*, 46(6), p. 1293. doi:10.2307/1913829.

Goldfeld, S. M., & Quandt, R. E. (1965). Some tests for homoscedasticity. *Econometrica*, 33(3), 540–541. <https://doi.org/10.2307/1913101>

Breusch, T.S. and Pagan, A.R. (1979) 'A simple test for heteroscedasticity and random coefficient variation', *Econometrica*, 47(5), p. 1287. doi:10.2307/1911963.

Newey, W.K. and West, K.D. (1987) 'A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix', *Econometrica*, 55(3), p. 703. doi:10.2307/1913610.

Quandt, R. E. (1960). "Tests of the hypothesis that a linear regression system obeys two separate regimes." *Journal of the American Statistical Association*, 55(290), 324-330.

Shen, D., Urquhart, A. and Wang, P. (2020) 'A three-factor pricing model for cryptocurrencies', *Finance Research Letters*, 34, p. 101248. doi:10.1016/j.frl.2019.07.021.

# Appendix: Python Code

October 13, 2024

## 0.1 Importing the Required Libraries

```
[1]: import pandas as pd
import numpy as np
import yfinance as yf
from fredapi import Fred
import statsmodels.api as sm
import statsmodels.formula.api as smf
import statsmodels.stats.api as sms
from datetime import datetime
import matplotlib as mpl
import matplotlib.pyplot as plt
import scipy.stats as stats
from scipy.stats import norm, f, chi2
import seaborn as sns
from tabulate import tabulate
from itertools import zip_longest as lzip
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.stattools import durbin_watson
from statsmodels.stats.diagnostic import het_white, het_breuschpagan, \
    ↳acorr_breusch_godfrey, acorr_ljungbox, linear_reset, breaks_cusumolsresid
from statsmodels.compat import lzip
import warnings
warnings.filterwarnings("ignore")
```

## 0.2 Part a) and b)

- a) Download the prices of 2 cryptocurrencies from September 2015 (or the oldest release) to August 2024 (in historical data). Transform them adequately to have log return  $\times 100$  ( $rt = \ln(P_t/P_{t-1}) \times 100$ ). Take the weekly frequency. Download the dataset from the Federal Reserve of St. Louis' website, <https://fred.stlouisfed.org> (SP500 and risk free interest rate Federal Funds Effective Rate) for the same sample period. Take the weekly frequency (ending Sunday).

```
[2]: start_date = pd.to_datetime('2015-08-31')
end_date = pd.to_datetime('2024-08-31')

#using yfinance to import crypto data directly from Yahoo Finance
```

```

crypto_currencies = yf.download('BTC-USD ETH-USD', start=start_date,
    ↪end=end_date, progress = False)
crypto_currencies = crypto_currencies['Close']

BTC = crypto_currencies['BTC-USD']
ETH = crypto_currencies['ETH-USD']

BTC = BTC.resample('W-SUN').last()
ETH = ETH.resample('W-SUN').last()

```

Downloading SP500 and Risk-free rate (Federal Funds Effective Rate)

```

[3]: #Utilising the FRED API to retrieve data directly from the Federal Reserve of St.
    ↪ Louis
fred = Fred(api_key='XXXXXXXXXXXXXXXXXXXXXXXXXXXX')

# (wesu) is a paramater which we can use to specify Weekly frequency, ending
    ↪ Sunday
SP500 = pd.DataFrame(fred.get_series('SP500', start_date, end_date, frequency =
    ↪'wesu'))
SP500 = SP500.rename(columns={0:"SP500"})

FFER = pd.DataFrame(fred.get_series('DFF', start_date, end_date, frequency =
    ↪'wesu'))
FFER = FFER.rename(columns={0:"FFER"})

```

```

[4]: BTC = pd.concat([BTC, SP500, FFER], axis = 1)
    ETH = pd.concat([ETH, SP500, FFER], axis = 1)

```

```

[5]: def compute_log_returns(df, crypto_col, sp500_col, ffr_col):
    df_copy = df.copy()
    df_copy[crypto_col] = np.log(df_copy[crypto_col]).diff() * 100
    ↪#Transformation to log returns
    df_copy[sp500_col] = np.log(df_copy[sp500_col]).diff() * 100
    result_df = df_copy[[crypto_col, sp500_col, ffr_col]].dropna()
    return result_df

BTC = compute_log_returns(BTC, 'BTC-USD', 'SP500', 'FFER')
ETH = compute_log_returns(ETH, 'ETH-USD', 'SP500', 'FFER')

```

### 0.3 Part c)

```

[6]: def capm_linear_reg(df, column, constant = True):

    df['ex_ret_asset'] = df[column] - df['FFER']
    df['ex_ret_mkt'] = df['SP500'] - df['FFER']

```

```

if constant:
    model = smf.ols(formula = 'ex_ret_asset~ex_ret_mkt',data=df).fit()
    df['y_hat'] = model.predict(df[['ex_ret_mkt']])
    df['residuals'] = df['ex_ret_asset'] - df['y_hat'] #residual calculation

else:
    model = smf.ols(formula = 'ex_ret_asset~ex_ret_mkt-1',data=df).fit()
    df['y_hat_no_const'] = model.predict(df[['ex_ret_mkt']])
    df['residuals_no_const'] = df['ex_ret_asset'] - df['y_hat_no_const']
    ↪#residual calculation

print('\nOLS Results for', column,':\n', model.summary())

return model

BTC_reg = capm_linear_reg(BTC, 'BTC-USD')
ETH_reg = capm_linear_reg(ETH, 'ETH-USD')

```

OLS Results for BTC-USD :

```

                        OLS Regression Results
=====
Dep. Variable:          ex_ret_asset    R-squared:                0.086
Model:                  OLS            Adj. R-squared:          0.084
Method:                 Least Squares   F-statistic:              44.14
Date:                  Sun, 13 Oct 2024 Prob (F-statistic):       8.52e-11
Time:                  14:30:44         Log-Likelihood:          -1728.4
No. Observations:      469             AIC:                    3461.
Df Residuals:          467             BIC:                    3469.
Df Model:               1
Covariance Type:       nonrobust
=====

```

|            | coef   | std err | t     | P> t  | [0.025 | 0.975] |
|------------|--------|---------|-------|-------|--------|--------|
| Intercept  | 1.1738 | 0.522   | 2.249 | 0.025 | 0.148  | 2.200  |
| ex_ret_mkt | 1.1436 | 0.172   | 6.644 | 0.000 | 0.805  | 1.482  |

```

=====
Omnibus:                19.837    Durbin-Watson:           1.934
Prob(Omnibus):          0.000    Jarque-Bera (JB):        40.723
Skew:                   -0.211    Prob(JB):                1.44e-09
Kurtosis:               4.380    Cond. No.                3.66
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Results for ETH-USD :

### OLS Regression Results

```
=====
Dep. Variable:          ex_ret_asset    R-squared:                0.107
Model:                  OLS            Adj. R-squared:          0.105
Method:                 Least Squares   F-statistic:             42.48
Date:                  Sun, 13 Oct 2024 Prob (F-statistic):       2.46e-10
Time:                  14:30:44         Log-Likelihood:          -1396.0
No. Observations:      355             AIC:                   2796.
Df Residuals:          353             BIC:                   2804.
Df Model:               1
Covariance Type:       nonrobust
=====
```

```
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      1.4391      0.806       1.787      0.075     -0.145      3.023
ex_ret_mkt      1.5398      0.236       6.518      0.000      1.075      2.004
=====
```

```
=====
Omnibus:                 33.407    Durbin-Watson:           1.930
Prob(Omnibus):            0.000    Jarque-Bera (JB):        128.966
Skew:                    -0.264    Prob(JB):                9.89e-29
Kurtosis:                 5.905    Cond. No.                4.31
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[7]: def plot_regression_and_residuals(crypto1, crypto2, robust=False, constant=True,
    ↪save=True):
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))

    # Define plot characteristics for each crypto asset
    for idx, df in enumerate([crypto1, crypto2]):
        title_suffix = '(Robust)' if robust else ''
        const_label = '' if constant else ' (No Constant)'
        y_hat_key = 'y_hat_robust' if robust else 'y_hat_no_const' if not
    ↪constant else 'y_hat'
        residuals_key = 'residuals_robust' if robust else 'residuals_no_const'
    ↪if not constant else 'residuals'

        # First row - scatter plot of actual vs predicted
        row, col = 0, idx
        axes[row, col].scatter(df['ex_ret_mkt'], df['ex_ret_asset'],
    ↪color='navy', marker='o', s=10, alpha=0.7, label='Observed Data')
        axes[row, col].plot(df['ex_ret_mkt'], df[y_hat_key], color='darkgray',
    ↪linewidth=2, label='Regression Line')
```

```

    # Titles and labels
    axes[row, col].set_title(f'{df.columns[0]} - Predicted vs Actual_
→{title_suffix} {const_label}', fontsize=14)
    axes[row, col].set_xlabel('Market Excess Return')
    axes[row, col].set_ylabel('Asset Excess Return')
    axes[row, col].legend()

    # Second row - histogram of residuals
    row = 1
    axes[row, col].hist(df[residuals_key], bins=40, color='mediumpurple',
→edgecolor='black', alpha=0.5, density=True, label='Residuals')

    # Overlay normal distribution curve
    mean_resid, std_resid = df[residuals_key].mean(), df[residuals_key].std()
    x_values = np.linspace(df[residuals_key].min(), df[residuals_key].max(),
→100)
    normal_pdf = stats.norm.pdf(x_values, mean_resid, std_resid)
    axes[row, col].plot(x_values, normal_pdf, color='thistle', linewidth=2.
→5, linestyle='-', alpha=1, label='Normal Distribution')
    axes[row, col].fill_between(x_values, 0, normal_pdf, color='thistle',
→alpha=0.5)

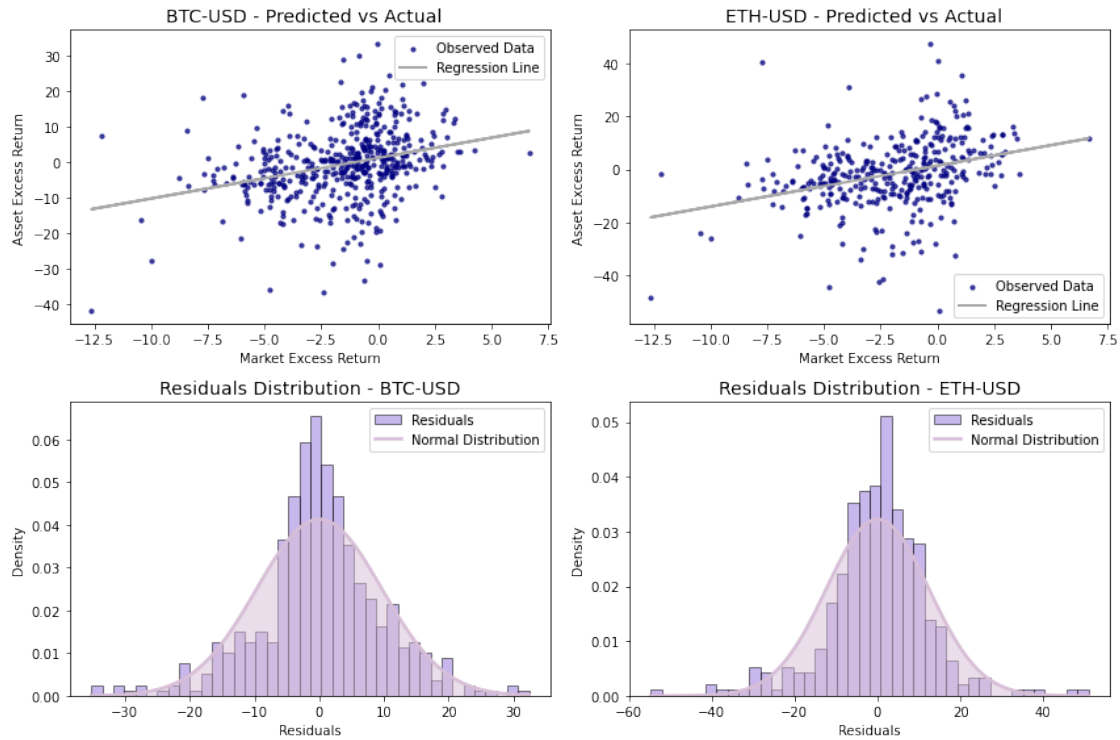
    # Titles and labels for residuals plot
    axes[row, col].set_title(f'Residuals Distribution - {df.columns[0]}',
→fontsize=14)
    axes[row, col].set_xlabel('Residuals')
    axes[row, col].set_ylabel('Density')
    axes[row, col].legend()

    # Adjust layout and optionally save the plot
    plt.tight_layout()
    if save:
        file_suffix = 'robust' if robust else 'nonrobust'
        const_suffix = '' if constant else '_noConstant'
        plt.savefig(f"/Users/jamesmurphy/FIN30200/
→regression_residuals_{file_suffix}{const_suffix}.jpeg", dpi=300,
→bbox_inches='tight')

    # Display the plot
    plt.show()

```

```
[8]: plot_regression_and_residuals(BTC, ETH, robust=False, constant=True, save=True)
```



### Tests for Non-normality

```
[9]: def jarque_bera_test(reg, crypto_name):
    crypto = crypto_name[:3]
    residuals = reg.resid
    jb_stat, jb_pvalue = stats.jarque_bera(residuals)

    print("\n--- Jarque-Bera Test ({}) ---".format(crypto))
    print(f"Test Statistic: {round(jb_stat, 3)}")
    print(f"p-value: {round(jb_pvalue, 3)}")
    print("-----")

    jarque_bera_test(BTC_reg, 'BTC')
    jarque_bera_test(ETH_reg, 'ETH')
```

```
--- Jarque-Bera Test (BTC) ---
Test Statistic: 40.723
p-value: 0.0
-----
```

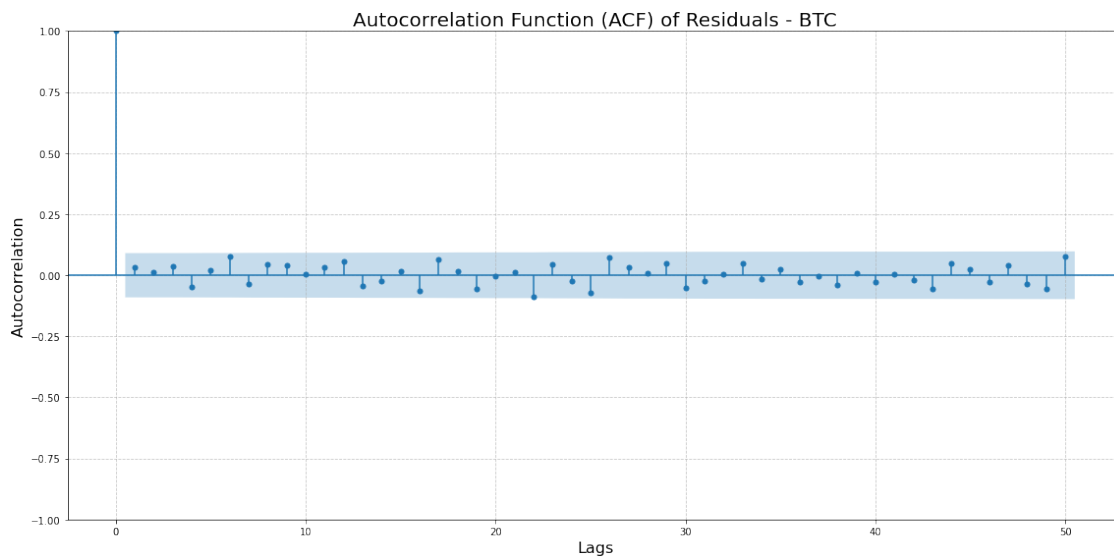
```
--- Jarque-Bera Test (ETH) ---
Test Statistic: 128.966
p-value: 0.0
```

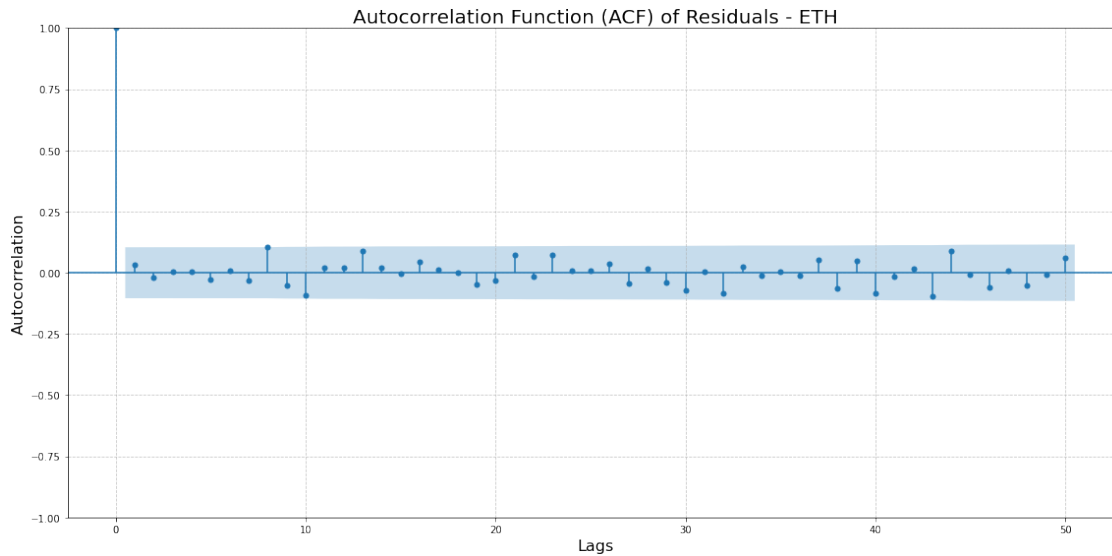


### Testing for Serial Correlation/ Autocorrelation

```
[10]: def plot_autocorrelation(reg, crypto_name, lags=50):  
  
    # Extract the residuals from the regression result  
    residuals = reg.resid  
    fig, ax = plt.subplots(figsize=(16, 8))  
    sm.graphics.tsa.plot_acf(residuals, lags=lags, alpha=0.05, ax=ax) # 95%  
    ↪ confidence interval (5% significance level)  
  
    ax.set_title(f"Autocorrelation Function (ACF) of Residuals - {crypto_name}",  
    ↪ fontsize=20)  
    ax.set_xlabel("Lags", fontsize=16)  
    ax.set_ylabel("Autocorrelation", fontsize=16)  
  
    # Add grid lines for better readability  
    ax.grid(True, linestyle='--', alpha=0.7)  
  
    # Show plot  
    plt.tight_layout()  
    plt.show()
```

```
[11]: plot_autocorrelation(BTC_reg, 'BTC')  
plot_autocorrelation(ETH_reg, 'ETH')
```





```
[12]: def q_stats(reg, crypto_name, nlags=50):

    series = reg.resid
    # Run the autocorrelation function (ACF) with qstat=True for Ljung-Box
    ↪ Q-Statistic
    acf_results = sm.tsa.acf(series, qstat=True, nlags=nlags, alpha=0.05)

    # Create a dictionary to hold Q-Stats and p-values
    q_stats = {'Q-Stats': acf_results[2], 'p-values': acf_results[3]}

    # Convert the dictionary to a DataFrame
    df_q_stats = pd.DataFrame(q_stats)

    # Add the 'Lag' column
    df_q_stats['Lag'] = range(1, nlags + 1)

    # Reorder the columns to show 'Lag' first
    df_q_stats = df_q_stats[['Lag', 'Q-Stats', 'p-values']]

    # Return the DataFrame
    return df_q_stats

q_stats(BTC_reg, 'BTC', nlags = 50)
```

```
[12]:
```

|   | Lag | Q-Stats  | p-values |
|---|-----|----------|----------|
| 0 | 1   | 0.477445 | 0.489582 |
| 1 | 2   | 0.538844 | 0.763821 |
| 2 | 3   | 1.130827 | 0.769638 |

|    |    |           |          |
|----|----|-----------|----------|
| 3  | 4  | 2.166236  | 0.705215 |
| 4  | 5  | 2.370739  | 0.795824 |
| 5  | 6  | 5.103652  | 0.530589 |
| 6  | 7  | 5.775255  | 0.566223 |
| 7  | 8  | 6.702529  | 0.569041 |
| 8  | 9  | 7.461171  | 0.589220 |
| 9  | 10 | 7.464659  | 0.680970 |
| 10 | 11 | 8.008524  | 0.712540 |
| 11 | 12 | 9.473091  | 0.662079 |
| 12 | 13 | 10.472900 | 0.654882 |
| 13 | 14 | 10.752910 | 0.705329 |
| 14 | 15 | 10.866326 | 0.762011 |
| 15 | 16 | 12.860425 | 0.682934 |
| 16 | 17 | 15.014489 | 0.594438 |
| 17 | 18 | 15.145777 | 0.651934 |
| 18 | 19 | 16.724368 | 0.608535 |
| 19 | 20 | 16.734374 | 0.670148 |
| 20 | 21 | 16.820687 | 0.721917 |
| 21 | 22 | 20.808259 | 0.532614 |
| 22 | 23 | 21.825487 | 0.530807 |
| 23 | 24 | 22.114874 | 0.572410 |
| 24 | 25 | 24.571284 | 0.486594 |
| 25 | 26 | 27.088413 | 0.404708 |
| 26 | 27 | 27.615565 | 0.430993 |
| 27 | 28 | 27.668070 | 0.482137 |
| 28 | 29 | 28.894789 | 0.470556 |
| 29 | 30 | 30.275376 | 0.451617 |
| 30 | 31 | 30.565467 | 0.488254 |
| 31 | 32 | 30.574032 | 0.538715 |
| 32 | 33 | 31.741528 | 0.529687 |
| 33 | 34 | 31.894016 | 0.571220 |
| 34 | 35 | 32.184792 | 0.604734 |
| 35 | 36 | 32.575257 | 0.632266 |
| 36 | 37 | 32.578672 | 0.676382 |
| 37 | 38 | 33.315853 | 0.685717 |
| 38 | 39 | 33.369288 | 0.723941 |
| 39 | 40 | 33.713813 | 0.747864 |
| 40 | 41 | 33.717763 | 0.783065 |
| 41 | 42 | 33.891784 | 0.809205 |
| 42 | 43 | 35.430211 | 0.787189 |
| 43 | 44 | 36.708133 | 0.774209 |
| 44 | 45 | 36.998832 | 0.796127 |
| 45 | 46 | 37.413256 | 0.812514 |
| 46 | 47 | 38.235386 | 0.815213 |
| 47 | 48 | 38.973457 | 0.820450 |
| 48 | 49 | 40.619569 | 0.797258 |
| 49 | 50 | 43.873185 | 0.716417 |

### Breusch-Godfrey Test

```
[13]: def run_breusch_godfrey_test(reg, crypto_name, nlags=10):

    bg_test = acorr_breusch_godfrey(reg, nlags=nlags)

    lm_stat = bg_test[0] # LM Statistic
    p_value = bg_test[1] # p-value
    print(f"\n--- Breusch-Godfrey Test for {crypto_name} ---")
    print(f"Lagrange Multiplier (LM) Statistic: {lm_stat}")
    print(f"p-value: {p_value}")

    # Return the results
    return lm_stat, p_value

bg_test_btc = run_breusch_godfrey_test(BTC_reg, 'BTH')
bg_test_eth = run_breusch_godfrey_test(ETH_reg, 'ETH')
```

```
--- Breusch-Godfrey Test for BTH ---
Lagrange Multiplier (LM) Statistic: 7.238448548035821
p-value: 0.7027584798664216
```

```
--- Breusch-Godfrey Test for ETH ---
Lagrange Multiplier (LM) Statistic: 9.500509803104517
p-value: 0.4853507812328879
```

### Durbin-Watson Test

```
[14]: def durbin_watson(reg):

    durbin_watson_test = sms.durbin_watson(reg.resid)
    print('Durbin-Watson Statistic:', durbin_watson_test)
```

```
[15]: durbin_watson(BTC_reg)
```

```
Durbin-Watson Statistic: 1.933696825857483
```

```
[16]: durbin_watson(ETH_reg)
```

```
Durbin-Watson Statistic: 1.9296058612554603
```

### Checking for Heteroskedasticity

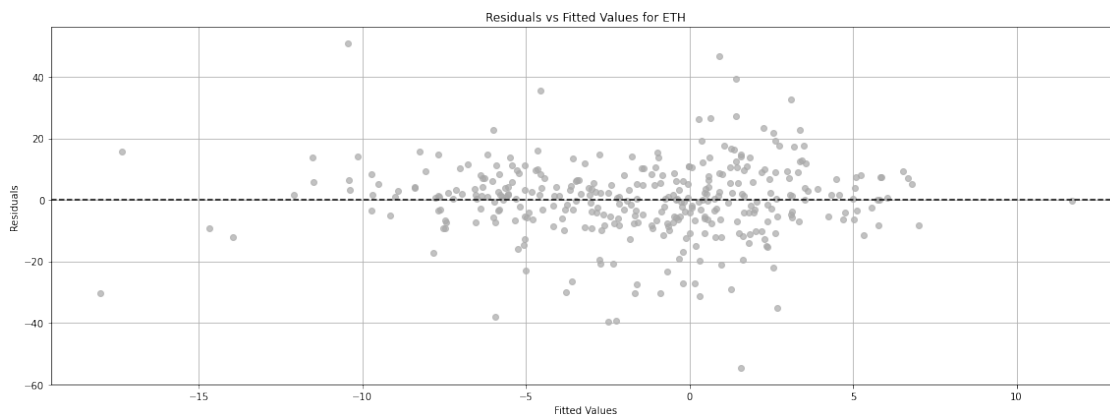
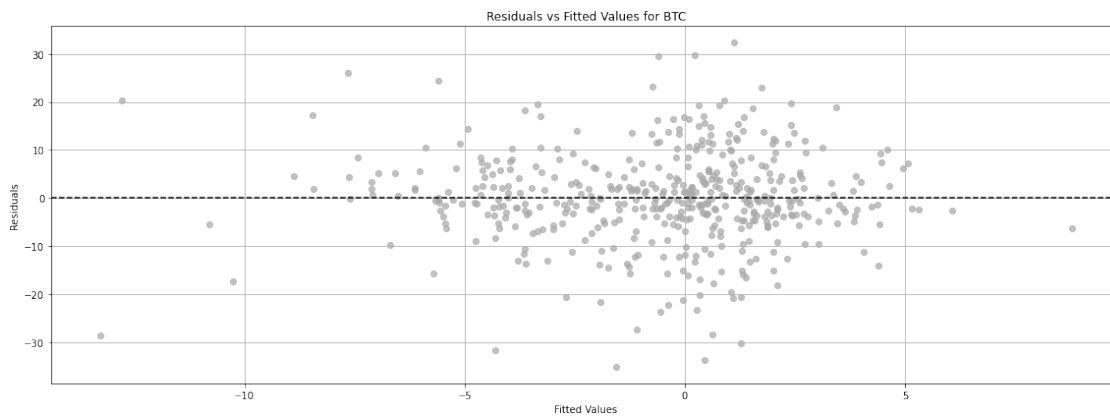
```
[17]: def plot_residuals_vs_fitted(reg, crypto_name):

    residuals = reg.resid
    fitted = reg.fittedvalues

    plt.figure(figsize=(16, 6))
    plt.scatter(fitted, residuals, color = 'darkgrey', alpha=0.7)
```

```
plt.axhline(y=0, color='black', linestyle='--')
plt.title(f'Residuals vs Fitted Values for {crypto_name}')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.grid(True)
plt.tight_layout()
plt.show()

plot_residuals_vs_fitted(BTC_reg, 'BTC')
plot_residuals_vs_fitted(ETH_reg, 'ETH')
```



## Breusch-Pagan Test

```
[18]: def breusch_pagan_test(reg):

    breuschpagan_name = ['Breusch-Pagan Lagrange Multiplier Statistic',
        ↳ 'p-value', 'f-value', 'f p-value']
    breuschpagan_test = sms.het_breuschpagan(reg.resid, reg.model.exog)
```

```
print(tabulate(lzip(breuschpagan_name, breuschpagan_test), headers=['Test',  
↪'Value'], tablefmt='pretty'))
```

```
breusch_pagan_test(BTC_reg)
```

```
breusch_pagan_test(ETH_reg)
```

| Test                                        | Value                |
|---------------------------------------------|----------------------|
| Breusch-Pagan Lagrange Multiplier Statistic | 1.059525205288807    |
| p-value                                     | 0.30332349958403415  |
| f-value                                     | 1.0573957533529281   |
| f p-value                                   | 0.30434220062716555  |
| Test                                        | Value                |
| Breusch-Pagan Lagrange Multiplier Statistic | 0.002152650256014854 |
| p-value                                     | 0.96299409689102     |
| f-value                                     | 0.002140535628732528 |
| f p-value                                   | 0.9631244620318543   |

## White Test

```
[19]: def white_test(reg):  
  
    white_test_name = ['White Test Lagrange Multiplier Statistic', 'p-value',  
↪'f-value', 'f p-value']  
    white_test_results = sms.het_white(reg.resid, reg.model.exog)  
    print(tabulate(lzip(white_test_name, white_test_results), headers=['Test',  
↪'Value'], tablefmt='pretty'))  
  
white_test(BTC_reg)  
white_test(ETH_reg)
```

| Test                                     | Value               |
|------------------------------------------|---------------------|
| White Test Lagrange Multiplier Statistic | 2.81691337334501    |
| p-value                                  | 0.24452036363797014 |
| f-value                                  | 1.4079035357947802  |
| f p-value                                | 0.24569435819549704 |
| Test                                     | Value               |
| White Test Lagrange Multiplier Statistic | 0.30646724708148276 |
| p-value                                  | 0.8579292659238876  |

|               |           |                     |
|---------------|-----------|---------------------|
|               | f-value   | 0.15206997169557726 |
|               | f p-value | 0.8589845761849855  |
| +-----+-----+ |           |                     |

### Goldfeld - Quandt Test

```
[20]: def goldfeld_quandt_test(model, crypto_name):

    goldfeldquandt_name = ['F-statistic', 'p-value']
    goldfeldquandt_test = sms.het_goldfeldquandt(model.resid, model.model.exog,
    ↪split=0.5)
    test_result = lzip(goldfeldquandt_name, goldfeldquandt_test)
    print(f"\n\nResults for {crypto_name}:")
    print(f"F-statistic: {test_result[0][1]}")
    print(f"p-value: {test_result[1][1]}")

goldfeld_quandt_test(BTC_reg, "BTC-USD")
goldfeld_quandt_test(ETH_reg, "ETH-USD")
```

Results for BTC-USD:  
F-statistic: 0.6074233920244703  
p-value: 0.9999204937343995

Results for ETH-USD:  
F-statistic: 0.490744882159583  
p-value: 0.9999984064024396

## 0.4 Part d)

```
[21]: def capm_linear_reg_robust_se(df, column, constant = True):

    df['ex_ret_asset'] = df[column] - df['FFER']
    df['ex_ret_mkt'] = df['SP500'] - df['FFER']

    if constant:
        model = smf.ols(formula = 'ex_ret_asset~ex_ret_mkt',data=df).
    ↪fit(cov_type='HAC', cov_kwds={'maxlags':12, 'use_correction':True})
        df['y_hat_robust'] = model.predict(df[['ex_ret_mkt']])
        df['residuals_robust'] = df['ex_ret_asset'] - df['y_hat_robust']
    ↪#residual calculation

    else:
        model = smf.ols(formula = 'ex_ret_asset~ex_ret_mkt-1',data=df).
    ↪fit(cov_type='HAC', cov_kwds={'maxlags':12, 'use_correction':True})
        df['y_hat_robust_no_const'] = model.predict(df[['ex_ret_mkt']])
```

```

df['residuals_robust_no_const'] = df['ex_ret_asset'] -
↳df['y_hat_robust_no_const'] #residual calculation

print('\nOLS Results for', column, ':\n', model.summary())

return model

BTC_reg_robust = capm_linear_reg_robust_se(BTC, 'BTC-USD')
ETH_reg_robust = capm_linear_reg_robust_se(ETH, 'ETH-USD')

```

OLS Results for BTC-USD :

```

                                OLS Regression Results
=====
Dep. Variable:                ex_ret_asset    R-squared:                0.086
Model:                        OLS            Adj. R-squared:           0.084
Method:                      Least Squares   F-statistic:             55.77
Date:                        Sun, 13 Oct 2024 Prob (F-statistic):       4.04e-13
Time:                        14:30:47        Log-Likelihood:          -1728.4
No. Observations:            469            AIC:                    3461.
Df Residuals:                467            BIC:                    3469.
Df Model:                    1
Covariance Type:             HAC
=====

               coef      std err          z      P>|z|      [0.025      0.975]
-----
Intercept      1.1738      0.572      2.051      0.040      0.052      2.296
ex_ret_mkt      1.1436      0.153      7.468      0.000      0.843      1.444
=====

Omnibus:                19.837    Durbin-Watson:           1.934
Prob(Omnibus):          0.000    Jarque-Bera (JB):        40.723
Skew:                   -0.211    Prob(JB):                1.44e-09
Kurtosis:               4.380    Cond. No.                3.66
=====

```

Notes:

[1] Standard Errors are heteroscedasticity and autocorrelation robust (HAC) using 12 lags and with small sample correction

OLS Results for ETH-USD :

```

                                OLS Regression Results
=====
Dep. Variable:                ex_ret_asset    R-squared:                0.107
Model:                        OLS            Adj. R-squared:           0.105
Method:                      Least Squares   F-statistic:             40.69
Date:                        Sun, 13 Oct 2024 Prob (F-statistic):       5.59e-10
Time:                        14:30:47        Log-Likelihood:          -1396.0
No. Observations:            355            AIC:                    2796.

```



Df Residuals: 353 BIC: 2804.  
Df Model: 1  
Covariance Type: HAC

|                | coef   | std err           | z        | P> z  | [0.025 | 0.975] |
|----------------|--------|-------------------|----------|-------|--------|--------|
| Intercept      | 1.4391 | 0.864             | 1.666    | 0.096 | -0.254 | 3.132  |
| ex_ret_mkt     | 1.5398 | 0.241             | 6.379    | 0.000 | 1.067  | 2.013  |
| Omnibus:       | 33.407 | Durbin-Watson:    | 1.930    |       |        |        |
| Prob(Omnibus): | 0.000  | Jarque-Bera (JB): | 128.966  |       |        |        |
| Skew:          | -0.264 | Prob(JB):         | 9.89e-29 |       |        |        |
| Kurtosis:      | 5.905  | Cond. No.         | 4.31     |       |        |        |

Notes:

[1] Standard Errors are heteroscedasticity and autocorrelation robust (HAC) using 12 lags and with small sample correction

Comparing Standard Errors

```
[22]: def check_standard_errors(df, crypto_name):
    # Non-robust model
    model = smf.ols(formula='ex_ret_asset ~ ex_ret_mkt', data=df).fit()

    # Robust model
    model_robust = smf.ols(formula='ex_ret_asset ~ ex_ret_mkt', data=df).
    →fit(cov_type='HAC', cov_kwds={'maxlags': 12})

    # Compare the standard errors
    print("\n\nNon-Robust Standard Errors ({}):".format(crypto_name))
    print(model.bse)

    print("\n\nRobust Standard Errors ({}):".format(crypto_name))
    print(model_robust.bse)
```

```
[23]: check_standard_errors(BTC, 'BTC')
check_standard_errors(ETH, 'ETH')
```

Non-Robust Standard Errors (BTC):

Intercept 0.522019  
ex\_ret\_mkt 0.172128  
dtype: float64

Robust Standard Errors (BTC):

Intercept 0.571124  
ex\_ret\_mkt 0.152817

dtype: float64

Non-Robust Standard Errors (ETH):

Intercept 0.805520

ex\_ret\_mkt 0.236246

dtype: float64

Robust Standard Errors (ETH):

Intercept 0.861393

ex\_ret\_mkt 0.240704

dtype: float64

## 0.5 Part e)

```
[24]: hypotheses_test = "ex_ret_mkt = 1"

regression_models = {'BTC': BTC_reg_robust, 'ETH': ETH_reg_robust}

for crypto_name, reg in regression_models.items():
    t_stat = reg.t_test(hypotheses_test)
    print(f"\nHypothesis Test for {crypto_name}:")
    print(t_stat)
```

Hypothesis Test for BTC:

| Test for Constraints |        |         |       |       |        |        |
|----------------------|--------|---------|-------|-------|--------|--------|
|                      | coef   | std err | z     | P> z  | [0.025 | 0.975] |
| c0                   | 1.1436 | 0.153   | 0.938 | 0.348 | 0.843  | 1.444  |

Hypothesis Test for ETH:

| Test for Constraints |        |         |       |       |        |        |
|----------------------|--------|---------|-------|-------|--------|--------|
|                      | coef   | std err | z     | P> z  | [0.025 | 0.975] |
| c0                   | 1.5398 | 0.241   | 2.236 | 0.025 | 1.067  | 2.013  |

```
[25]: regression_models = {'BTC': BTC_reg_robust, 'ETH': ETH_reg_robust}

for crypto_name, reg in regression_models.items():
    conf_interval = reg.conf_int(alpha=0.05) # 95% confidence interval
    print(f"\n95% Confidence Interval for {crypto_name}:")
    print(conf_interval)
```

95% Confidence Interval for BTC:

|            | 0        | 1        |
|------------|----------|----------|
| Intercept  | 0.052026 | 2.295578 |
| ex_ret_mkt | 0.843473 | 1.443787 |

95% Confidence Interval for ETH:

|            | 0         | 1        |
|------------|-----------|----------|
| Intercept  | -0.254011 | 3.132140 |
| ex_ret_mkt | 1.066742  | 2.012953 |

## 0.6 Part f)

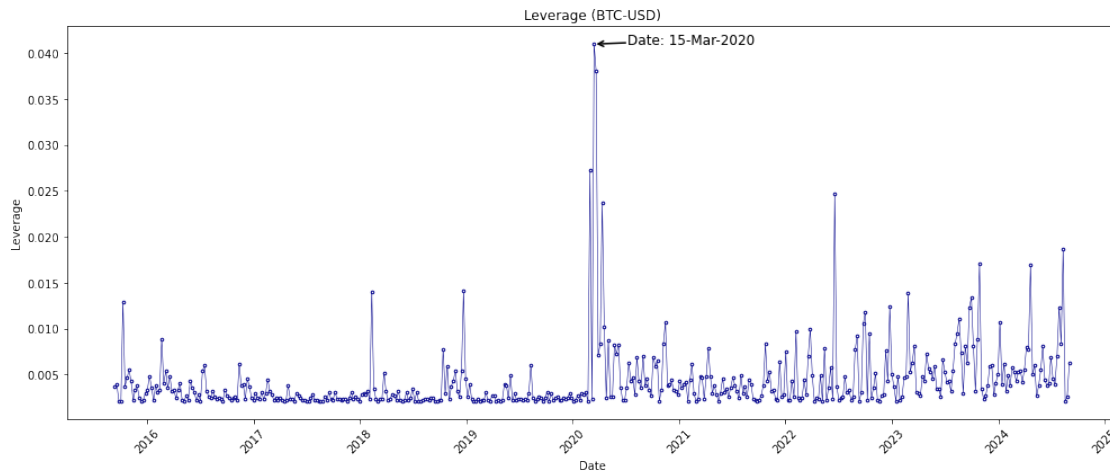
Identifying a suitable timepoint for splitting the data in order to effectively implement the Chow Test to look for parameter instabilities

```
[26]: def leverage_influence(model, crypto_df):  
  
    # Calculate Leverage (hat values)  
    infl = model.get_influence()  
    leverage = infl.hat_matrix_diag  
    leverage_df = pd.DataFrame({'Date': crypto_df.index, "Leverage": leverage})  
    idx_max = leverage_df["Leverage"].idxmax()  
    max_leverage = leverage_df["Leverage"].max()  
    date_max = crypto_df.index[idx_max]  
  
    plt.figure(figsize=(14, 6))  
    plt.plot(leverage_df['Date'], leverage_df["Leverage"],  
→markeredgecolor='darkblue', marker='.', markersize=5,  
        markerfacecolor='white', linestyle='-', color='darkblue',  
→linewidth=0.5)  
  
    plt.annotate(f"Date: {date_max.strftime('%d-%b-%Y')}", (date_max,  
→max_leverage),  
        xytext=(30, 0), textcoords='offset points',  
        arrowprops=dict(arrowstyle='->', lw=1.5, color='black',  
→shrinkA=0),  
        fontsize=12, color='black')  
    plt.title(f"Leverage ({crypto_df.columns[0]})")  
    plt.xlabel('Date')  
    plt.ylabel("Leverage")  
    plt.xticks(rotation=45)  
    plt.tight_layout()  
    plt.savefig(f"leverageInfluence_{crypto_df.columns[0]}.jpeg", dpi=300,  
→bbox_inches='tight')  
    print(f"Observation with highest leverage for {crypto_df.columns[0]}:  
→{date_max}")
```

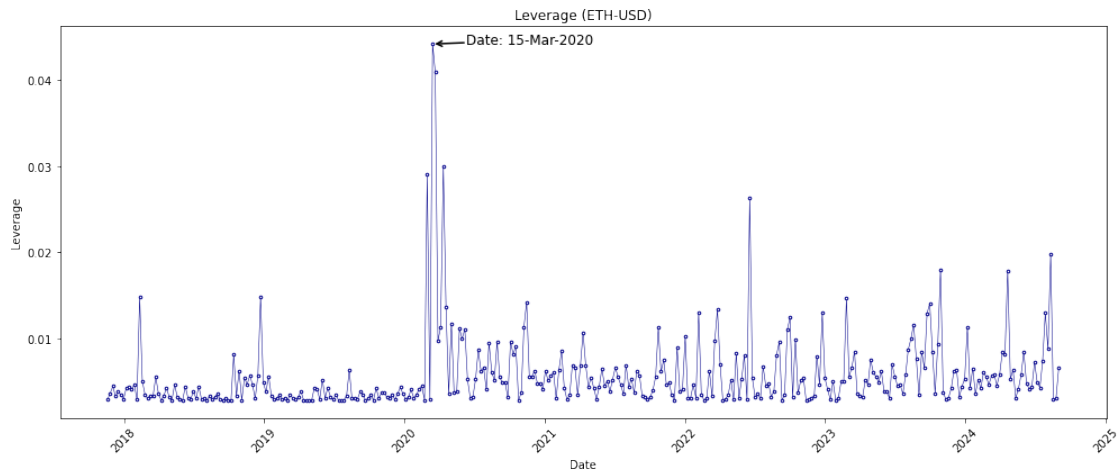
```
plt.show()
```

```
leverage_influence(BTC_reg, BTC)  
leverage_influence(ETH_reg, ETH)
```

Observation with highest leverage for BTC-USD: 2020-03-15 00:00:00



Observation with highest leverage for ETH-USD: 2020-03-15 00:00:00



Identifying the worst weekly performance of BTC and ETH

```
[27]: def worst_performance(df, asset):  
    print("\n\n{} Worst Recorded Performance:".format(asset))  
    print("Timestamp: {}".format(df[asset].idxmin()))  
    print("Return: {}".format(df[asset].min()))
```

```
worst_performance(BTC, 'BTC-USD')
worst_performance(ETH, 'ETH-USD')
worst_performance(BTC, 'SP500')
```

BTC-USD Worst Recorded Performance:

Timestamp: 2020-03-15 00:00:00

Return: -40.78907804416563

ETH-USD Worst Recorded Performance:

Timestamp: 2021-05-23 00:00:00

Return: -53.09684855669783

SP500 Worst Recorded Performance:

Timestamp: 2020-03-22 00:00:00

Return: -12.004422381000612

Identifying the five worst weekly performances of ETH

```
[28]: lowest_values = ETH['ETH-USD'].nsmallest(5)
      print(lowest_values.index, lowest_values.values)
```

```
DatetimeIndex(['2021-05-23', '2020-03-15', '2018-11-25', '2018-09-09',
               '2018-02-04'],
              dtype='datetime64[ns]', freq=None) [-53.09684856 -47.17300413
-41.90662295 -40.2023037  -40.06509107]
```

```
[29]: lowest_values = BTC['SP500'].nsmallest(5)
      print(lowest_values.index, lowest_values.values)
```

```
DatetimeIndex(['2020-03-22', '2020-03-15', '2020-03-01', '2022-06-19',
               '2018-02-11'],
              dtype='datetime64[ns]', freq=None) [-12.00442238 -11.53529337
-8.88269614  -8.73767975  -6.28250294]
```

```
[30]: #Date ranges for the COVID-19 pandemic period
      highlight_start = '2020-02-01'
      highlight_end = '2020-05-01'

      plt.figure(figsize=(12, 8))

      # Plot BTC weekly excess returns
      plt.plot(BTC.index, BTC['BTC-USD'], label='BTC Excess Returns', color='black',
               ↪linewidth=1.5)
```

```

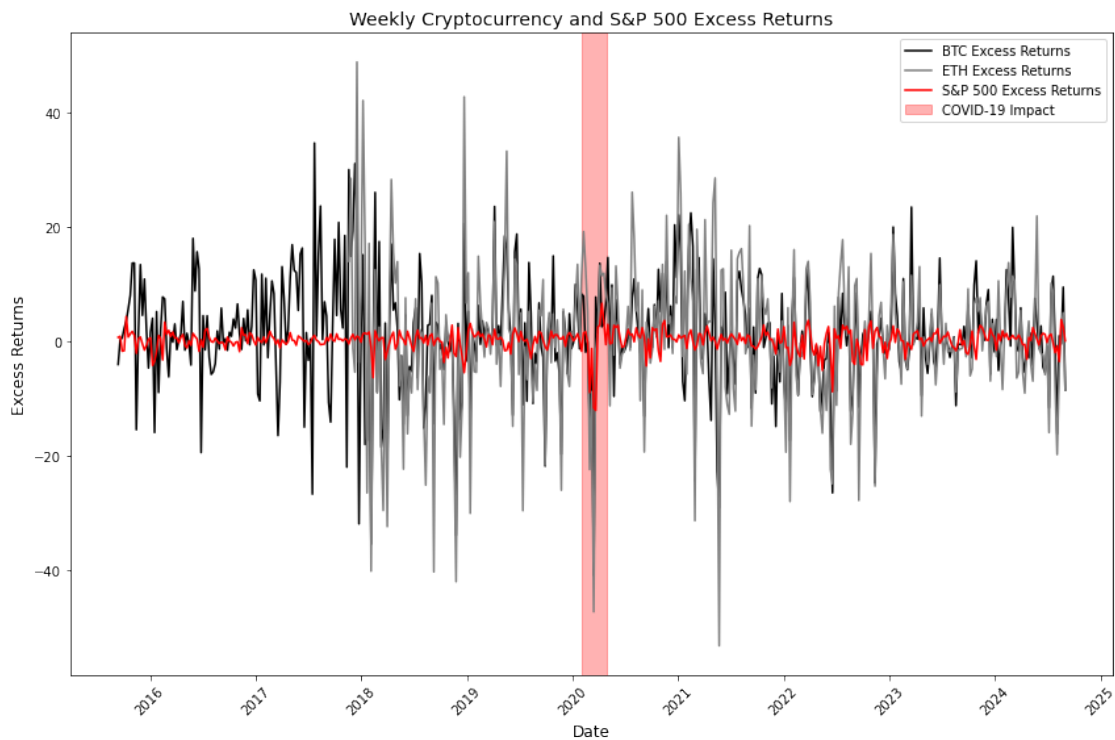
# Plot ETH weekly excess returns
plt.plot(ETH.index, ETH['ETH-USD'], label='ETH Excess Returns', color='grey',
        linewidth=1.5)

# Plot SP500 weekly excess returns
plt.plot(BTC.index, BTC['SP500'], label='S&P 500 Excess Returns', color='red',
        linewidth=1.5)

# Highlight a period of intense volatility (e.g., during the COVID-19 pandemic)
plt.axvspan(highlight_start, highlight_end, color='red', alpha=0.3,
            label='COVID-19 Impact')

plt.title('Weekly Cryptocurrency and S&P 500 Excess Returns', fontsize=14)
plt.xlabel('Date', fontsize=12)
plt.ylabel('Excess Returns', fontsize=12)
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()

```



```

[31]: # Date obtained by using Leverage (Hat Values)
# Sunday of the week that the pandemic was declared
split_date = '2020-03-15'

```

```

BTC_sample1 = BTC[BTC.index < split_date]
BTC_sample2 = BTC[BTC.index >= split_date]

ETH_sample1 = ETH[ETH.index <= split_date]
ETH_sample2 = ETH[ETH.index > split_date]

# Running the regression for BTC (pre-COVID and post-COVID)
BTC_reg_sample1 = capm_linear_reg(BTC_sample1, 'BTC-USD')
BTC_reg_sample2 = capm_linear_reg(BTC_sample2, 'BTC-USD')

# Running the regression for ETH (pre-COVID and post-COVID)
ETH_reg_sample1 = capm_linear_reg(ETH_sample1, 'ETH-USD')
ETH_reg_sample2 = capm_linear_reg(ETH_sample2, 'ETH-USD')

```

OLS Results for BTC-USD :

```

                                OLS Regression Results
=====
Dep. Variable:                  ex_ret_asset    R-squared:                  0.029
Model:                            OLS          Adj. R-squared:             0.025
Method:                  Least Squares        F-statistic:                 7.003
Date:                Sun, 13 Oct 2024        Prob (F-statistic):         0.00869
Time:                14:30:49                Log-Likelihood:            -891.28
No. Observations:                235          AIC:                       1787.
Df Residuals:                    233          BIC:                       1793.
Df Model:                            1
Covariance Type:                nonrobust
=====

```

|            | coef   | std err | t     | P> t  | [0.025 | 0.975] |
|------------|--------|---------|-------|-------|--------|--------|
| Intercept  | 1.4094 | 0.833   | 1.692 | 0.092 | -0.232 | 3.051  |
| ex_ret_mkt | 1.0952 | 0.414   | 2.646 | 0.009 | 0.280  | 1.911  |

```

=====
Omnibus:                        8.808    Durbin-Watson:              1.965
Prob(Omnibus):                  0.012    Jarque-Bera (JB):           13.286
Skew:                          -0.216    Prob(JB):                   0.00130
Kurtosis:                      4.082    Cond. No.                   2.59
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Results for BTC-USD :

```

                                OLS Regression Results
=====
Dep. Variable:                  ex_ret_asset    R-squared:                  0.154
Model:                            OLS          Adj. R-squared:             0.151

```

```

Method:                Least Squares    F-statistic:                42.32
Date:                  Sun, 13 Oct 2024  Prob (F-statistic):        4.72e-10
Time:                  14:30:49          Log-Likelihood:            -829.87
No. Observations:      234              AIC:                      1664.
Df Residuals:          232              BIC:                      1671.
Df Model:              1
Covariance Type:       nonrobust

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      0.8547      0.658      1.299      0.195     -0.441      2.151
ex_ret_mkt      1.1292      0.174      6.505      0.000      0.787      1.471
=====

Omnibus:                11.862    Durbin-Watson:                1.848
Prob(Omnibus):           0.003    Jarque-Bera (JB):            20.443
Skew:                    -0.264    Prob(JB):                    3.64e-05
Kurtosis:                4.349    Cond. No.                     4.62
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Results for ETH-USD :

#### OLS Regression Results

```

=====
Dep. Variable:          ex_ret_asset    R-squared:                0.063
Model:                  OLS            Adj. R-squared:            0.055
Method:                 Least Squares   F-statistic:              8.083
Date:                  Sun, 13 Oct 2024 Prob (F-statistic):        0.00525
Time:                  14:30:49          Log-Likelihood:            -506.83
No. Observations:      122              AIC:                      1018.
Df Residuals:          120              BIC:                      1023.
Df Model:              1
Covariance Type:       nonrobust

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      1.0555      1.917      0.551      0.583     -2.741      4.852
ex_ret_mkt      1.9789      0.696      2.843      0.005      0.601      3.357
=====

Omnibus:                8.518    Durbin-Watson:                1.876
Prob(Omnibus):           0.014    Jarque-Bera (JB):            16.087
Skew:                    0.175    Prob(JB):                    0.000321
Kurtosis:                4.744    Cond. No.                     4.00
=====

```

Notes:



[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Results for ETH-USD :

| OLS Regression Results |                  |                   |                     |          |        |        |
|------------------------|------------------|-------------------|---------------------|----------|--------|--------|
| Dep. Variable:         | ex_ret_asset     |                   | R-squared:          | 0.161    |        |        |
| Model:                 | OLS              |                   | Adj. R-squared:     | 0.157    |        |        |
| Method:                | Least Squares    |                   | F-statistic:        | 44.24    |        |        |
| Date:                  | Sun, 13 Oct 2024 |                   | Prob (F-statistic): | 2.08e-10 |        |        |
| Time:                  | 14:30:49         |                   | Log-Likelihood:     | -874.13  |        |        |
| No. Observations:      | 233              |                   | AIC:                | 1752.    |        |        |
| Df Residuals:          | 231              |                   | BIC:                | 1759.    |        |        |
| Df Model:              | 1                |                   |                     |          |        |        |
| Covariance Type:       | nonrobust        |                   |                     |          |        |        |
|                        | coef             | std err           | t                   | P> t     | [0.025 | 0.975] |
| Intercept              | 1.8930           | 0.809             | 2.339               | 0.020    | 0.298  | 3.488  |
| ex_ret_mkt             | 1.4524           | 0.218             | 6.651               | 0.000    | 1.022  | 1.883  |
| Omnibus:               | 40.432           | Durbin-Watson:    | 1.936               |          |        |        |
| Prob(Omnibus):         | 0.000            | Jarque-Bera (JB): | 136.830             |          |        |        |
| Skew:                  | -0.669           | Prob(JB):         | 1.94e-30            |          |        |        |
| Kurtosis:              | 6.508            | Cond. No.         | 4.53                |          |        |        |

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

### Chow Test

```
[32]: def chow_test(fitted_full_model, fitted_first_half_model,
    ↪fitted_second_half_model, crypto_name):

    # Get the residual sum of squares (RSS) for each model
    RSS_full = sum(fitted_full_model.resid ** 2) # Pooled model RSS
    RSS_first_half = sum(fitted_first_half_model.resid ** 2)
    RSS_second_half = sum(fitted_second_half_model.resid ** 2)

    # Number of parameters in the model (k)
    k = len(fitted_full_model.params)

    # Number of observations in each subset
    n_first_half = len(fitted_first_half_model.resid)
    n_second_half = len(fitted_second_half_model.resid)
    n_full = len(fitted_full_model.resid)
```

```

# Calculate the Chow test F-statistic
numerator = (RSS_full - (RSS_first_half + RSS_second_half)) / k
denominator = (RSS_first_half + RSS_second_half) / (n_first_half +
↪n_second_half - 2 * k)
F_stat = numerator / denominator

# Calculate the p-value using the F-distribution
df1 = k # degrees of freedom for numerator
df2 = (n_first_half + n_second_half - 2 * k) # degrees of freedom for
↪denominator
p_value = 1 - stats.f.cdf(F_stat, df1, df2) # Right-tail p-value for
↪F-distribution

# Print the F-statistic and p-value
print("\n\nChow Test Results for {}".format(crypto_name))
print(f"F-statistic: {F_stat}")
print(f"p-value: {p_value}")

# Conclusion based on p-value
if p_value < 0.05:
    print(f"There is evidence of a structural break in the {crypto_name}
↪data (p-value < 0.05).")
else:
    print(f"There is no significant structural break detected in the
↪{crypto_name} data (p-value >= 0.05).")

chow_test(BTC_reg, BTC_reg_sample1, BTC_reg_sample2, "BTC")
chow_test(ETH_reg, ETH_reg_sample1, ETH_reg_sample2, "ETH")

```

Chow Test Results for BTC

F-statistic: 0.21947418158223111

p-value: 0.8030240159824168

There is no significant structural break detected in the BTC data (p-value >= 0.05).

Chow Test Results for ETH

F-statistic: 1.2512944490211657

p-value: 0.28740734523969524

There is no significant structural break detected in the ETH data (p-value >= 0.05).

**Quandt-Andrews Test**

```
[33]: def quandt_andrews_test(model, crypto_name):

    # Perform the Quandt-Andrews test using residuals
    qa_test = sm.stats.diagnostic.breaks_cusumolsresid(model.resid)

    # Display the results
    print(f"\nQuandt-Andrews Test Results for {crypto_name}:")
    print(f"Test statistic: {qa_test[0]}")
    print(f"p-value: {qa_test[1]}")
    if qa_test[1] < 0.05:
        print(f"There is a significant structural break in the {crypto_name} ↵
↵data (p-value < 0.05).")
    else:
        print(f"There is no significant structural break in the {crypto_name} ↵
↵data (p-value >= 0.05).")
```

```
[34]: quandt_andrews_test(BTC_reg, 'BTC-USD')
quandt_andrews_test(ETH_reg, 'ETH-USD')
```

Quandt-Andrews Test Results for BTC-USD:  
Test statistic: 1.3050649077564134  
p-value: 0.06631904874316534  
There is no significant structural break in the BTC-USD data (p-value >= 0.05).

Quandt-Andrews Test Results for ETH-USD:  
Test statistic: 0.6863182548875724  
p-value: 0.7338745057062599  
There is no significant structural break in the ETH-USD data (p-value >= 0.05).

## 0.7 Part g)

Repeat point c) without the constant.

```
[37]: BTC_reg_no_constant = capm_linear_reg(BTC, 'BTC-USD', constant = False)
      ETH_reg_no_constant = capm_linear_reg(ETH, 'ETH-USD', constant = False)
```

OLS Results for BTC-USD :

```

                                OLS Regression Results
=====
Dep. Variable:                  ex_ret_asset    R-squared (uncentered):
0.080
Model:                          OLS          Adj. R-squared (uncentered):
0.078
Method:                        Least Squares   F-statistic:
40.70
Date:                          Sun, 13 Oct 2024 Prob (F-statistic):
```

```

4.27e-10
Time:                  14:30:51   Log-Likelihood:
-1730.9
No. Observations:      469   AIC:
3464.
Df Residuals:          468   BIC:
3468.
Df Model:              1
Covariance Type:      nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
ex_ret_mkt      0.9428      0.148      6.380      0.000      0.652      1.233
=====
Omnibus:          20.726   Durbin-Watson:          1.904
Prob(Omnibus):    0.000   Jarque-Bera (JB):          42.841
Skew:            -0.224   Prob(JB):          4.98e-10
Kurtosis:         4.411   Cond. No.          1.00
=====

```

Notes:

- [1]  $R^2$  is computed without centering (uncentered) since the model does not contain a constant.
- [2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Results for ETH-USD :

```

                                OLS Regression Results
=====
Dep. Variable:          ex_ret_asset   R-squared (uncentered):
0.113
Model:                  OLS   Adj. R-squared (uncentered):
0.110
Method:                 Least Squares   F-statistic:
44.92
Date:                  Sun, 13 Oct 2024   Prob (F-statistic):
8.11e-11
Time:                  14:30:51   Log-Likelihood:
-1397.6
No. Observations:      355   AIC:
2797.
Df Residuals:          354   BIC:
2801.
Df Model:              1
Covariance Type:      nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----

```

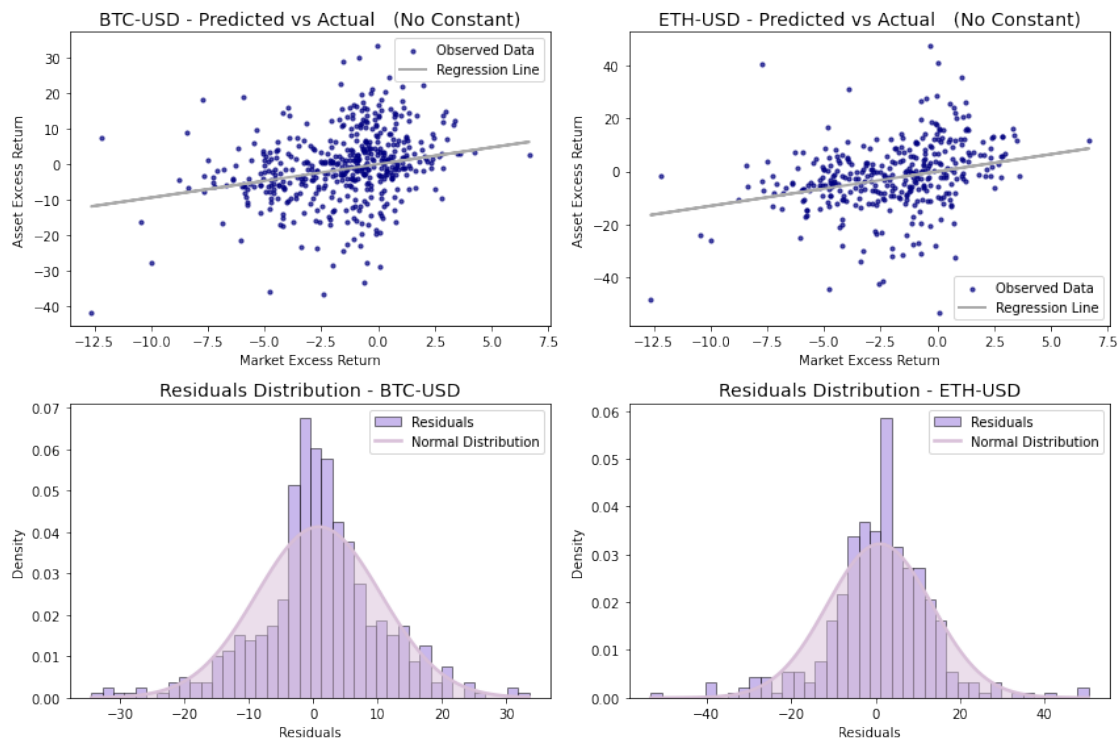
```
-----
ex_ret_mkt      1.2958      0.193      6.702      0.000      0.916      1.676
=====
Omnibus:                32.833      Durbin-Watson:                1.908
Prob(Omnibus):           0.000      Jarque-Bera (JB):             124.182
Skew:                    -0.263      Prob(JB):                     1.08e-27
Kurtosis:                5.849      Cond. No.                     1.00
=====
```

Notes:

[1]  $R^2$  is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[38]: plot_regression_and_residuals(BTC, ETH, robust=False, constant=False, save=True)
```



### Likelihood Ratio Test

```
[39]: def log_likelihood_test(model_with_const, model_without_const, crypto_name):

    # Perform the Likelihood Ratio Test
    lr_test = model_with_const.compare_lr_test(model_without_const)
```

```

# Extract LR statistic and p-value
lr_stat = lr_test[0]
p_value = lr_test[1]

# Print the results
print(f"\nLikelihood Ratio Test for {crypto_name}:\n")
print(f"LR Test Statistic: {lr_stat}")
print(f"p-value: {p_value}")

if p_value < 0.05:
    print(f"\nThe model with a constant provides a better fit for_
↳{crypto_name} (p-value < 0.05). "
        "You should include the constant in the model.\n")
else:
    print(f"\nThe model without a constant is sufficient for {crypto_name}_
↳(p-value >= 0.05). "
        "Including the constant does not significantly improve the model.
↳\n")

```

```

[40]: log_likelihood_test(BTC_reg, BTC_reg_no_constant, 'BTC')
log_likelihood_test(ETH_reg, ETH_reg_no_constant, 'ETH')

```

Likelihood Ratio Test for BTC:

LR Test Statistic: 5.05046692053611  
p-value: 0.02461930314567836

The model with a constant provides a better fit for BTC (p-value < 0.05). You should include the constant in the model.

Likelihood Ratio Test for ETH:

LR Test Statistic: 3.1952536705466628  
p-value: 0.0738523118277696

The model without a constant is sufficient for ETH (p-value >= 0.05). Including the constant does not significantly improve the model.

## RESET Test

```

[41]: reset_test_with_const = linear_reset(BTC_reg, power=2)
print(f"RESET Test with constant p-value: {reset_test_with_const.pvalue}")

# Perform the RESET test on the model without a constant
reset_test_without_const = linear_reset(BTC_reg_no_constant, power=2)

```

```
print(f"RESET Test without constant p-value: {reset_test_without_const.pvalue}")
```

RESET Test with constant p-value: 0.8397613063157051

RESET Test without constant p-value: 0.6865872860234747

### AIC vs BIC

```
[42]: def aic_bic(model_with_constant, model_no_constant, crypto_name):

    aic_with_const = model_with_constant.aic
    bic_with_const = model_with_constant.bic

    aic_without_const = model_no_constant.aic
    bic_without_const = model_no_constant.bic

    print("\n\nResults for {}".format(crypto_name))
    print(f"AIC with constant: {aic_with_const}, without constant: {
    ↪aic_without_const}")
    print(f"BIC with constant: {bic_with_const}, without constant: {
    ↪bic_without_const}")
```

```
[43]: aic_bic(BTC_reg, BTC_reg_no_constant, 'BTC')
      aic_bic(ETH_reg, ETH_reg_no_constant, 'ETH')
```

Results for BTC:

AIC with constant: 3460.8119520750915, without constant: 3463.8624189956276

BIC with constant: 3469.113157611984, without constant: 3468.013021764074

Results for ETH:

AIC with constant: 2796.0062162295444, without constant: 2797.201469900091

BIC with constant: 2803.750451808495, without constant: 2801.0735876895665

### Residuals

```
[44]: def plot_residuals(df, residuals_with_const, residuals_without_const,
    ↪crypto_name, ax, color1, color2):

    # Residuals with constant
    sns.histplot(df[residuals_with_const], bins=50, kde=False, color=color1,
    ↪stat='density', edgecolor='black', alpha=0.6, label='With Constant', ax=ax)
    mu_with, std_with = stats.norm.fit(df[residuals_with_const])
    x = np.linspace(ax.get_xlim()[0], ax.get_xlim()[1], 100)
    p_with = stats.norm.pdf(x, mu_with, std_with)
    ax.plot(x, p_with, color1, linewidth=2, label='Normal Dist (With Const)')

    # Residuals without constant
```

```

sns.histplot(df[residuals_without_const], bins=50, kde=False, color=color2,
stat='density', edgecolor='black', alpha=0.6, label='Without Constant', ax=ax)
mu_without, std_without = stats.norm.fit(df[residuals_without_const])
p_without = stats.norm.pdf(x, mu_without, std_without)
ax.plot(x, p_without, color2, linewidth=2, label='Normal Dist (Without
Const)')

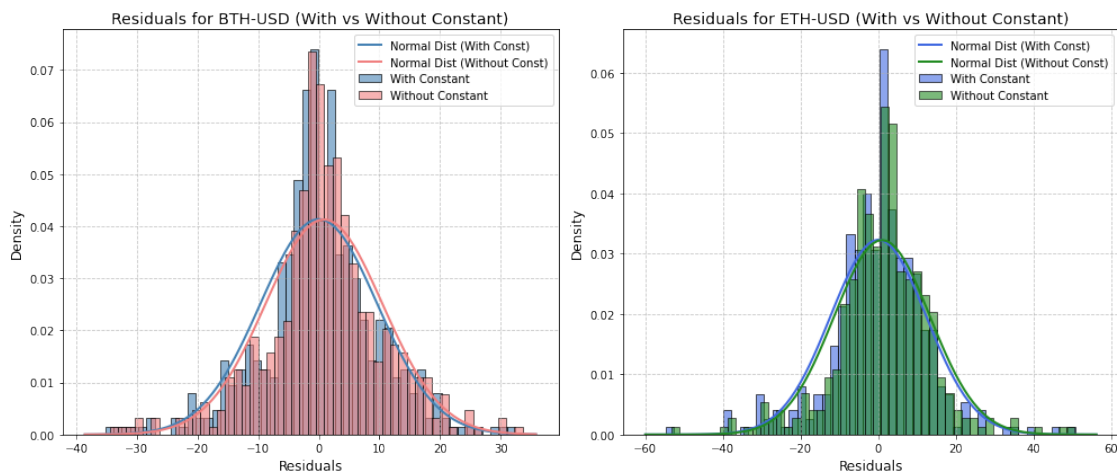
ax.set_title(f"Residuals for {crypto_name} (With vs Without Constant)",
fontsize=14)
ax.set_xlabel("Residuals", fontsize=12)
ax.set_ylabel("Density", fontsize=12)
ax.grid(True, linestyle='--', alpha=0.7)
ax.legend()

def plot_residuals_comparison(df1, residuals_with_const1,
residuals_without_const1, crypto_name1,
df2, residuals_with_const2,
residuals_without_const2, crypto_name2):

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(14, 6))
plot_residuals(df1, residuals_with_const1, residuals_without_const1,
crypto_name1, axes[0], 'steelblue', 'lightcoral')
plot_residuals(df2, residuals_with_const2, residuals_without_const2,
crypto_name2, axes[1], 'royalblue', 'forestgreen')
plt.tight_layout()
plt.show()

plot_residuals_comparison(BTC, 'residuals', 'residuals_no_const', 'BTH-USD',
ETH, 'residuals', 'residuals_no_const', 'ETH-USD')

```





```
[45]: plt.figure(figsize=(12, 6))

plt.title("Weekly Excess Returns",)

BTC['ex_ret_asset'].plot(label='BTC Excess Returns', color = 'darkblue', alpha=0.5,
    ↳linestyle='-', linewidth=2)

ETH['ex_ret_asset'].plot(label='ETH Excess Returns', color = 'lightblue',
    ↳alpha=0.5, linestyle='-', linewidth=2)

BTC['ex_ret_mkt'].plot(label='Market Excess Returns', color='black',
    ↳linestyle='-', linewidth=2)

plt.legend(loc='upper left', frameon=True, title='Assets')
plt.xlabel("Date")
plt.ylabel("Excess Returns")

plt.tight_layout()
plt.savefig(r"WeeklyExcessReturns.jpeg", dpi=300, bbox_inches='tight')

plt.show()
```

